

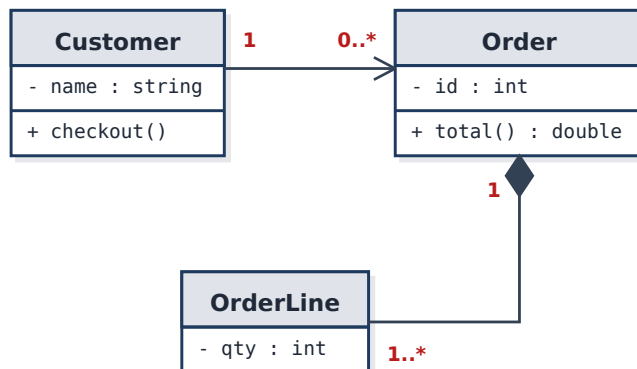
A COMPLETE BEGINNER'S GUIDE

UML Diagrams

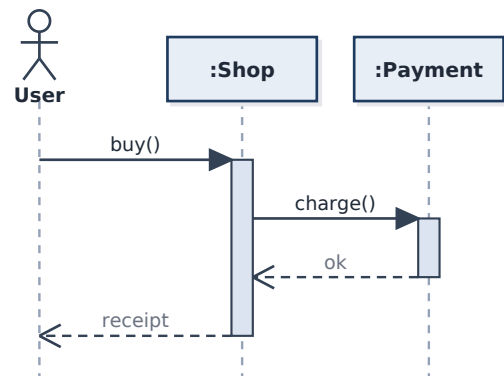
Class Diagrams & Sequence Diagrams

Notation, relationships, messages and fragments, explained step by step

Class diagram



Sequence diagram



- Every class-diagram element: classes, visibility, multiplicity and all 7 relationship types
- Every sequence-diagram element: lifelines, 8 message types, 7 fragments
- 4 full worked examples, built step by step from a problem statement
- C++ code for every diagram (compiled and tested), Q&A, exercises, cheat sheet

Contents

1. Introduction to UML	6
1.1 What is UML?	6
1.2 Why use UML diagrams?	6
1.3 A short history	6
1.4 The 14 UML diagram types	7
1.5 Class diagram vs sequence diagram in one picture	7
1.6 Tools	7
Part A • Class Diagrams	9
2. Class Notation	10
2.1 Anatomy of a class box	10
2.2 Visibility	10
2.3 Attribute syntax	11
2.4 Operation syntax	11
2.5 Static and abstract members	12
2.6 Interfaces, abstract classes and enumerations	12
2.7 Levels of detail	12
2.8 From class box to C++	13
3. Relationships: How Classes Connect	15
3.1 All relationships at a glance	15
3.2 Association	15
3.3 Multiplicity	16
3.4 Navigability (directed association)	17
3.5 Reflexive (self) association	18
3.6 Association class	18
3.7 Aggregation (hollow diamond)	19
3.8 Composition (filled diamond)	20
3.9 Aggregation vs composition	21
3.10 Generalization (inheritance)	22
3.11 Realization (implementing an interface)	23
3.12 Dependency	24
3.13 Choosing the right relationship	24
4. Drawing a Class Diagram Step by Step	26
4.1 The problem statement	26
4.2 Step 1: find the candidate classes (nouns)	26

4.3 Step 2: attributes and operations (verbs)	27
4.4 Step 3: relationships	27
4.5 Step 4: multiplicities	27
4.6 The finished diagram	27
4.7 The diagram as C++	28
5. Worked Example: Online Shopping	31
6. Class Diagram Tips and Common Mistakes	33
6.1 Common mistakes	33
6.2 Good practice	33
6.3 Class diagrams as text: PlantUML	33
Part B · Sequence Diagrams	35
7. Sequence Diagram Basics	36
7.1 When to use one	36
7.2 Anatomy of a sequence diagram	36
7.3 Participants	36
7.4 Lifelines and activation bars	37
7.5 Message types	37
7.6 Synchronous vs asynchronous	38
7.7 Return messages	38
7.8 Self messages	39
7.9 Creating and destroying objects	39
8. Combined Fragments: Conditions, Loops and More	41
8.1 alt: if / else	41
8.2 opt: if without else	41
8.3 loop: repetition	42
8.4 par: parallel	42
8.5 break: early exit	43
8.6 ref: reuse another diagram	43
8.7 All interaction operators	44
9. Worked Examples	46
9.1 ATM cash withdrawal	46
9.2 Online checkout	47
10. Drawing a Sequence Diagram Step by Step	51
10.1 The method	51
10.2 Common mistakes	51

10.3 Sequence diagrams as text: PlantUML	51
11. Class vs Sequence Diagrams	53
Keeping the two consistent	53
12. Exam and Interview Questions	54
13. Practice Exercises	56
14. Cheat Sheet and Glossary	57
Class diagram notation	57
Sequence diagram notation	58
Glossary	59

How to use this guide

This guide teaches the two UML diagrams you will use most often, and that are asked about most in exams and interviews:

- **Class diagrams** (Part A) show the *structure* of a system: which classes exist, what data and operations each one has, and how they are connected.
- **Sequence diagrams** (Part B) show *behaviour over time*: which objects send which messages to each other, and in what order, to carry out one scenario.

You only need basic object-oriented ideas (class, object, inheritance, interface). Every diagram element comes with a picture, a plain-English explanation, and in most cases the **C++ code it turns into**, so you can see that a diagram is just a picture of real code.

Tip

Draw every diagram yourself on paper as you read. UML is a visual language: you learn it by drawing it, exactly as you learn a programming language by typing it.

Work through Part A before Part B. Sequence diagrams use the classes defined in class diagrams.

How to read the diagrams in this guide

Colour / style	Meaning
Red bold numbers next to a line	Multiplicity (how many)
<i>Blue italic</i> words next to a line	Role names or guard conditions
<i>Black italic</i> word in the middle of a line, with ►	Association name and the direction to read it
Pink labels with a dot and thin line	Annotations explaining the diagram. They are not part of UML; you would not draw them yourself
Grey code boxes	C++ code; every snippet was compiled with g++ to make sure it is correct

1. Introduction to UML

1.1 What is UML?

UML (Unified Modeling Language) is a standard *visual* language for describing software systems. It gives everyone the same set of symbols, so a diagram drawn by a developer in Bengaluru can be read correctly by a developer in Berlin.

UML is **not** a programming language and it is not tied to any language. The same class diagram can be implemented in C++, Java, Python or C#. It is a way to *think about and communicate* a design before (and after) you write the code.

Real-life analogy

Before a building is built, an architect draws a **floor plan** (what rooms exist and how they connect) and engineers draw **wiring and plumbing diagrams** (how electricity and water flow). A class diagram is software's floor plan. A sequence diagram is its wiring diagram: it shows how requests flow between parts.

1.2 Why use UML diagrams?

Reason	What it means in practice
Communicate	A diagram explains a design to teammates, managers or clients in minutes. Reading 2,000 lines of code to understand the same thing takes hours.
Design before coding	Mistakes found on paper cost minutes to fix. The same mistake found after the code is written can cost days.
See the big picture	Code shows one file at a time. A class diagram shows the whole system on one page.
Document	New team members learn the system from diagrams far faster than from code alone.
Standard language	Because the symbols are standardised, there is no confusion over what a box or an arrow means.
Exams and interviews	UML is part of almost every software engineering course, and system design interviews regularly ask you to sketch class and sequence diagrams.

1.3 A short history

In the early 1990s there were dozens of competing diagram notations. Three of the best-known methodologists, **Grady Booch**, **James Rumbaugh** and **Ivar Jacobson** (nicknamed "the three amigos"), worked together at Rational Software to merge their methods into one. The result was adopted as a standard by the **Object Management Group (OMG)** in 1997. UML 2.0 (2005) was a major revision; the current version is **UML 2.5.1** (2017), and UML is also published as the international standard ISO/IEC 19505.

1.4 The 14 UML diagram types

UML 2.5 defines 14 diagram types in two families. **Structural** diagrams show what a system is made of; **behavioral** diagrams show what it does.

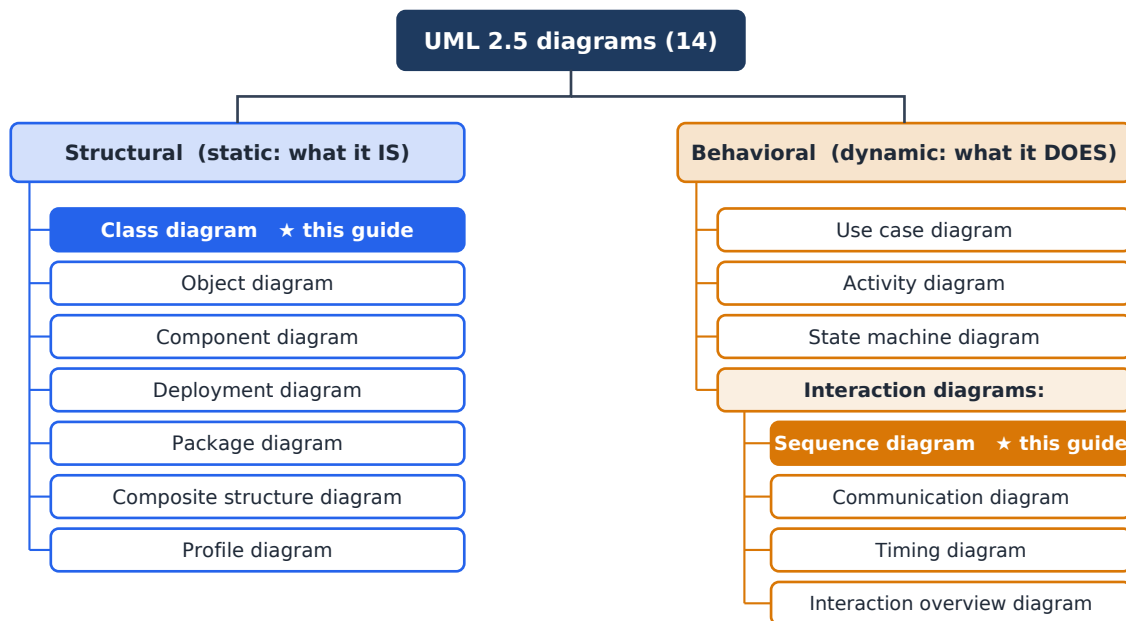


Figure 1.1 The UML diagram family. This guide covers the two highlighted diagrams.

In real projects a few diagrams do most of the work. Surveys of practising developers consistently find **class diagrams** and **sequence diagrams** among the most used, which is why this guide focuses on them.

1.5 Class diagram vs sequence diagram in one picture

	Class diagram	Sequence diagram
Answers	What classes exist and how are they connected?	Who talks to whom, in what order, for one scenario?
View	Static (structure)	Dynamic (behaviour over time)
Shows	Classes, attributes, operations, relationships	Objects, messages, order of calls, conditions and loops
Time	Not shown: true at all times	Flows from top to bottom
Like	A floor plan	A film storyboard of one scene

1.6 Tools

Pen and paper (or a whiteboard) is the best tool while learning and while designing. For clean, shareable diagrams, popular options are: **draw.io / diagrams.net** (free, drag and drop), **Lucidchart**, **StarUML** and **Visual Paradigm** (dedicated UML tools), and text-to-diagram tools such as **PlantUML** and **Mermaid**, where you type the diagram as text and the tool draws it. Chapters 6 and 10 show PlantUML versions of diagrams from this guide.

Key takeaways

- UML is a standard visual language for describing software designs.
- It is language-independent: a diagram can be implemented in any OO language.
- Structural diagrams show what a system IS; behavioral diagrams show what it DOES.
- Class diagram = static structure. Sequence diagram = messages over time.

Part A · Class Diagrams

Classes, their attributes and operations, and the seven kinds of relationship that connect them.

2. Class Notation

A class diagram is made of **class boxes** connected by **relationship lines**. This chapter covers the box; Chapter 3 covers the lines.

2.1 Anatomy of a class box

A class is drawn as a rectangle divided into three **compartments**: the name at the top, the attributes (data) in the middle, and the operations (functions) at the bottom.

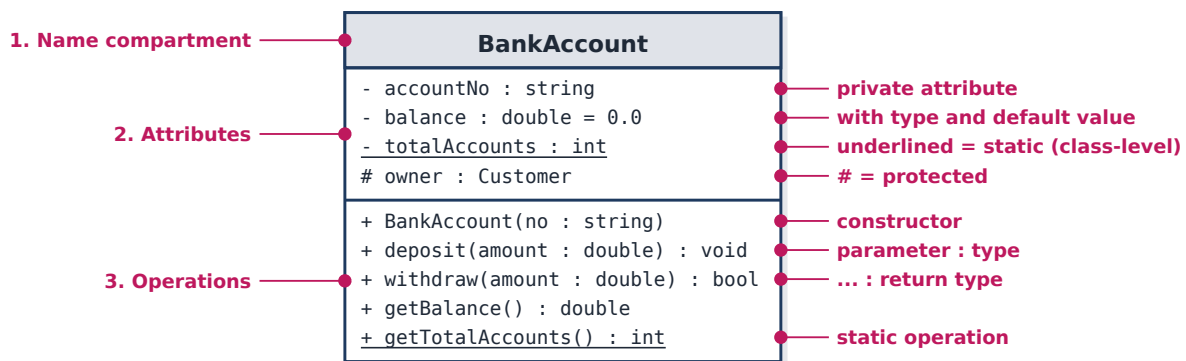


Figure 2.1 The three compartments of a class and what each part of a row means.

- **Name compartment:** the class name in bold, centred, starting with a capital letter (**BankAccount**, not **bankAccount**). Use a singular noun: **Book**, not **Books**.
- **Attribute compartment:** one line per data member.
- **Operation compartment:** one line per member function (UML calls functions **operations**; the code that implements them is a **method**).
- Compartments may be left empty or hidden entirely when the detail is not needed (see 2.7).

2.2 Visibility

The symbol at the start of each row says who may access that member. It maps directly to C++ access specifiers:

Symbol	UML name	C++ keyword	Who can access it
+	Public	<code>public</code>	Everyone
-	Private	<code>private</code>	Only the class itself
#	Protected	<code>protected</code>	The class and its subclasses
~	Package	(none in C++)	Classes in the same package (Java's default access); rarely used for C++ designs

Tip

A good default: attributes **private (-)**, operations that form the class's interface **public (+)**. If you see many **+** attributes in a diagram, encapsulation is probably missing.

2.3 Attribute syntax

The full form of an attribute is shown below. Only the **name** is required; add the other parts as the design becomes more detailed.

UML · attribute syntax

```
visibility name : Type [multiplicity] = defaultValue {property}
```

Part	Example	Meaning
visibility	-	private
name	balance	attribute name, lowerCamelCase
: Type	: double	data type; can be another class, e.g. : Customer
[multiplicity]	phones : string [0..3]	how many values it holds (a list of 0 to 3 phone numbers)
= default	= 0.0	initial value
{property}	{readOnly}	extra constraint; {readOnly} cannot change after creation (like const)
/ prefix	/ age : int	derived attribute: calculated from other data (age from date of birth), not stored

2.4 Operation syntax

UML · operation syntax

```
visibility name(param : Type, ...) : ReturnType {property}
```

Example	Meaning
+ getBalance() : double	public, no parameters, returns a double
+ withdraw(amount : double) : bool	one parameter, returns bool
- log(msg : string)	private; no return type written means it returns nothing (void)
+ getBalance() : double {query}	{query} = does not change the object (a const member function in C++)
+ swap(inout a : int, inout b : int)	parameter direction: in (default), out or inout (like a C++ reference parameter that is modified)

2.5 Static and abstract members

- **Static** (class-level) members are **underlined**. There is one copy shared by all objects, like `static int totalAccounts;` in C++.
- **Abstract** classes and operations are written in *italics*. Because italics are hard to see in handwriting, you can also write `{abstract}` under the class name. An abstract operation corresponds to a C++ pure virtual function `virtual ... = 0;`.

2.6 Interfaces, abstract classes and enumerations

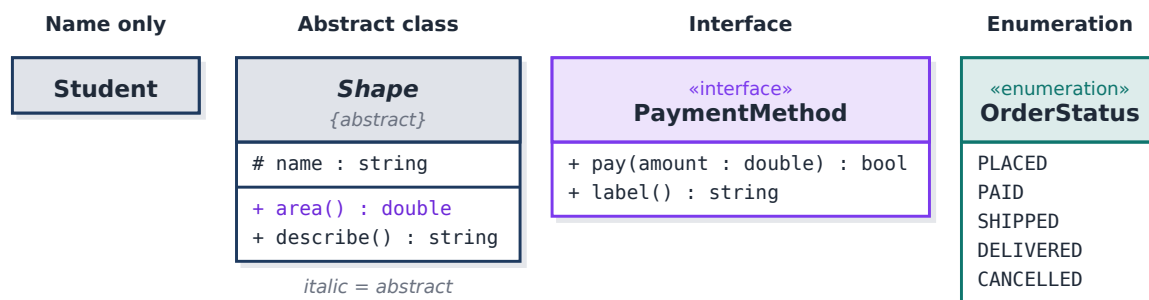


Figure 2.2 Special kinds of class box.

Kind	How it is drawn	C++ equivalent
Abstract class	Name in italics, optionally <code>{abstract}</code> ; abstract operations in italics	Class with at least one pure virtual function
Interface	<code>«interface»</code> above the name; usually only operations	Class with only pure virtual functions
Enumeration	<code>«enumeration»</code> above the name; values listed in the second compartment	<code>enum class OrderStatus { PLACED, PAID, ... };</code>
Other stereotypes	Any <code>«word»</code> above the name, such as <code>«entity»</code> , <code>«service»</code> , <code>«utility»</code>	Labels that classify the class; meaning agreed by the team

Note

The « » marks are called **guillemets**. In plain text you may write `<<interface>>` instead.

2.7 Levels of detail

You do not always draw every detail. The same class can be shown at three levels, depending on how far the design has progressed and who will read it:

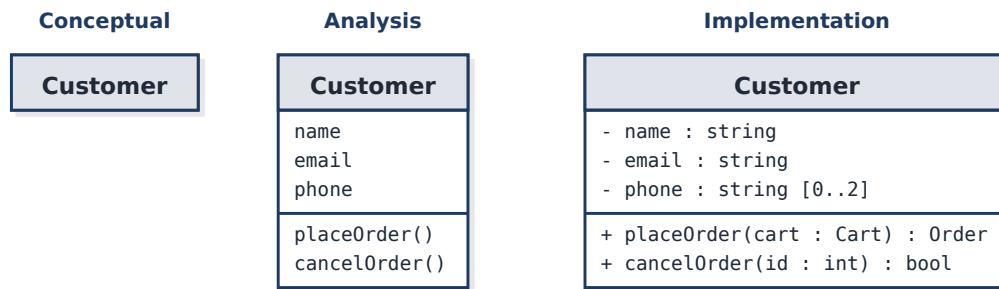


Figure 2.3 One class at three levels of detail.

Level	When	Shows
Conceptual	Early discussions with users and clients	Names and relationships only
Analysis	Understanding the problem	Attribute and operation names, no types
Implementation	Just before (or after) coding	Everything: visibility, types, parameters, return types

2.8 From class box to C++

Here is Figure 2.1 written as C++. Compare each row of the diagram with the matching line of code:

C++ · BankAccount.cpp

```
#include <string>
using namespace std;

class Customer;           // used as a type below

class BankAccount {
private:
    string accountNo;      // - accountNo : string
    double balance = 0.0;  // - balance : double = 0.0
    static int totalAccounts; // - totalAccounts : int (underlined)
protected:
    Customer* owner = nullptr; // # owner : Customer
public:
    BankAccount(string no) : accountNo(no) { totalAccounts++; }
    void deposit(double amount) { balance += amount; }
    bool withdraw(double amount) {
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }
    double getBalance() const { return balance; } // {query}
    static int getTotalAccounts() { return totalAccounts; }
};

int BankAccount::totalAccounts = 0;
```

Chapter 2 recap

- A class box has three compartments: name, attributes, operations.
- Visibility: + public, - private, # protected, ~ package.
- Attribute: - name : Type = default. Operation: + name(p : Type) : Return.
- Underlined = static. Italic (or {abstract}) = abstract.
- «interface» and «enumeration» mark special kinds of class.
- Show only the level of detail your reader needs.

3. Relationships: How Classes Connect

A class on its own does little. The value of a class diagram is in the **lines between classes**, which show how they depend on and work with each other. UML has seven common relationship types, each with its own line and arrowhead. Getting the arrowheads right is the single most important skill in class diagrams.

3.1 All relationships at a glance

Symbol	Name	Meaning	Typical C++
	Association	A and B are connected and work together ("knows about")	member pointer or reference
	Directed association	A knows about B; B does not know about A	only A has a member pointing to B
	Aggregation	A has B as a part, but B can exist on its own and can be shared	member pointer; A does not delete B
	Composition	A owns B; B lives and dies with A	member object stored by value
	Generalization (inheritance)	A is a kind of B (arrow points to the parent)	<code>class A : public B</code>
	Realization	A implements interface B	A overrides B's pure virtual functions
	Dependency	A uses B briefly (parameter, local variable, return type)	B appears only inside a function

Watch out

Watch where each decoration goes. The **diamond** goes at the **whole** (the owner). The **triangle** goes at the **parent** or **interface**. The **open arrowhead** points to the class being **known about or used**.

3.2 Association

An **association** is a structural link between two classes: objects of one class are connected to objects of the other for a meaningful period of time, usually because one holds a reference to the other. It is drawn as a **solid line**.



Figure 3.1 A fully decorated association.

An association line can carry several decorations. All are optional; add the ones that make the diagram clearer.

Decoration	In Figure 3.1	Meaning
Association name	<i>enrolls in</i>	A verb phrase describing the link
Reading direction ►	► after the name	Read it as "Student enrolls in Course", not the other way round
Role name	<i>students, courses</i>	The role a class plays in the link; often becomes the member variable name in code
Multiplicity	0..* and 1..6	How many objects take part (3.3)
Navigability	(none here)	Arrowhead showing which side knows about the other (3.4)

How to read an association

Read each end separately, **starting from the opposite class**:

- "One **Student** enrolls in **1..6** Courses" (read the number at the Course end).
- "One **Course** has **0..*** Students" (read the number at the Student end).

Watch out

The most common beginner mistake is putting the multiplicity at the wrong end. The number next to a class says **how many of THAT class** are linked to **one** object of the class at the other end.

3.3 Multiplicity

Notation	Meaning	Example
1	Exactly one	An order belongs to exactly 1 customer
0..1	Zero or one (optional)	A person has 0..1 passport
* or 0..*	Zero or more (any number)	A customer has 0..* orders
1..*	One or more (at least one)	An order has 1..* order lines
n..m	Between n and m	A student takes 1..6 courses
n	Exactly n	A car has 4 wheels
(nothing)	Not specified yet	Treat as unknown, not as 1

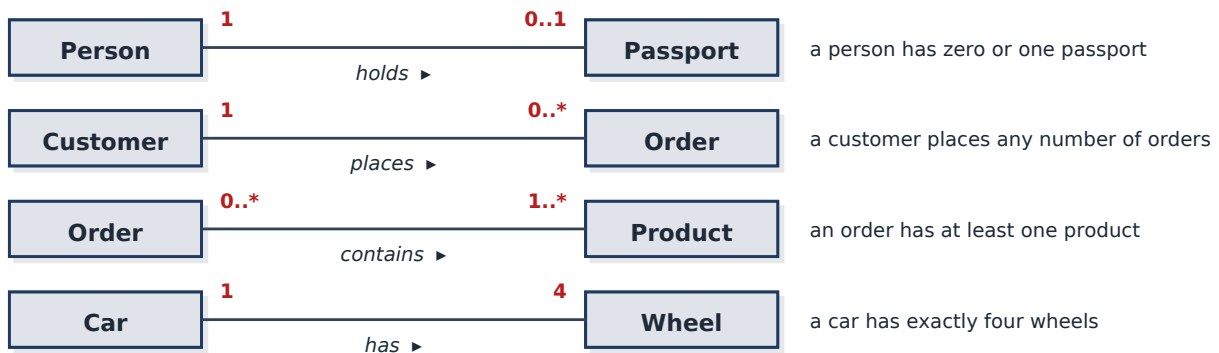


Figure 3.2 Common multiplicities in practice.

Two-question method

To find multiplicities, ask two questions, one for each direction:

- "For ONE Customer, how many Orders can there be, at minimum and at maximum?" Answer: 0 to many, so write **0..*** at the **Order** end.
- "For ONE Order, how many Customers?" Answer: exactly 1, so write **1** at the **Customer** end.

3.4 Navigability (directed association)

An **open arrowhead** on an association shows **navigability**: which class can reach the other. An arrow from Customer to Order means a Customer object can find its Orders, but an Order cannot find its Customer. With no arrowheads, the association is either navigable both ways or the direction simply has not been decided yet.

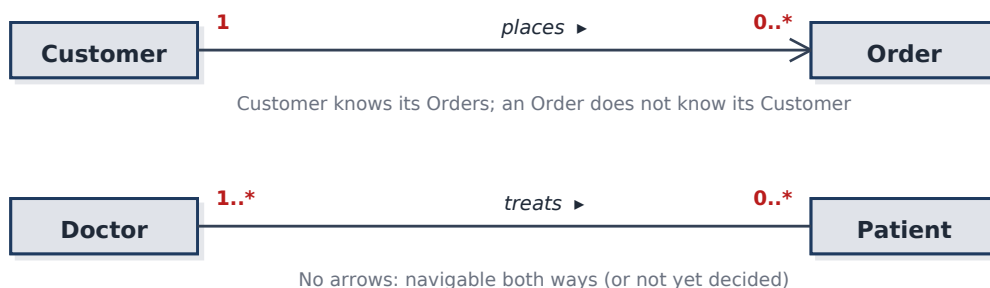


Figure 3.3 Directed vs undirected associations.

C++ · Customer.cpp

```
#include <vector>
using namespace std;

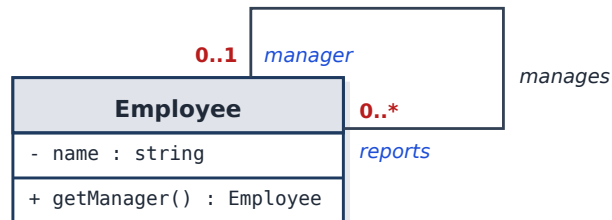
class Order { /* no pointer back to Customer */ };

class Customer {
    vector<Order*> orders;           // Customer ---> Order (1 to 0..*)
public:
    void place(Order* o) { orders.push_back(o); }
};
```

In code, navigability simply means **which class holds the pointer**. Two-way navigability means both classes hold pointers to each other, which is more work to keep consistent, so use it only when both directions are really needed.

3.5 Reflexive (self) association

A class can be associated with **itself**. An employee's manager is also an employee; a course can have other courses as prerequisites; a folder can contain folders. Role names are essential here, otherwise the two ends are indistinguishable.



Reflexive association: a class related to itself

Figure 3.4 An Employee has 0..1 manager and 0..* reports.

C++ · Employee.cpp

```
#include <string>
#include <vector>
using namespace std;

class Employee {
    string name;
    Employee* manager = nullptr;    // 0..1 manager
    vector<Employee*> reports;      // 0..* reports
public:
    Employee(string n) : name(n) {}
    void setManager(Employee* m) {
        manager = m;
        m->reports.push_back(this);
    }
    Employee* getManager() const { return manager; }
};
```

3.6 Association class

Sometimes data belongs to the **link itself**, not to either class. A grade is not a property of a student (they have many grades) or of a course (it has many students); it belongs to one *student in one course*. UML models this with an **association class**: a normal class box joined to the association line by a dashed line.

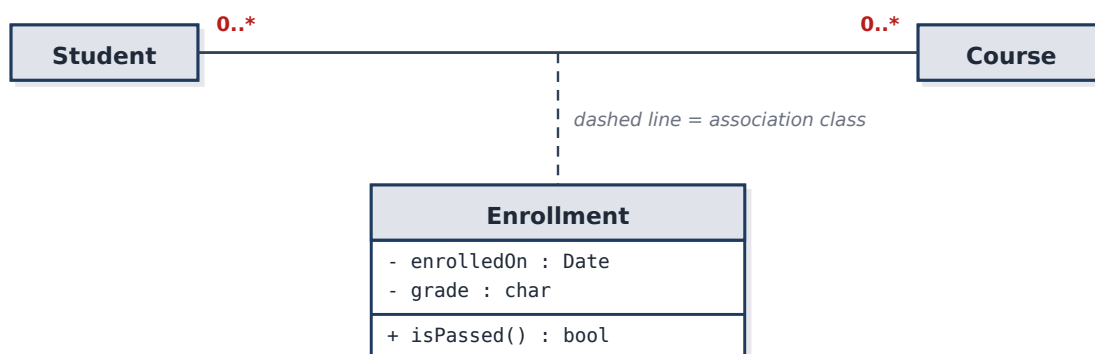


Figure 3.5 Enrollment is an association class holding the data of each Student–Course link.

C++ · Enrollment.cpp

```
#include <string>
using namespace std;

class Student { /* ... */ };
class Course { /* ... */ };

// The association class becomes a real class that links the two ends
// and stores the data that belongs to the LINK, not to either end.
class Enrollment {
    Student* student;
    Course* course;
    string enrolledOn;
    char grade = '-';
public:
    Enrollment(Student* s, Course* c, string date)
        : student(s), course(c), enrolledOn(date) {}
    void setGrade(char g) { grade = g; }
    bool isPassed() const { return grade != 'F' && grade != '-'; }
};
```

3.7 Aggregation (hollow diamond)

Aggregation is a special association meaning **whole-part**, where the part can exist independently of the whole and may even be shared between several wholes. It is drawn with a **hollow diamond at the whole's end**.



Figure 3.6 A department has professors; a playlist has songs.

A professor existed before joining the department and continues to exist if the department is closed. A song can be in many playlists at once. In C++ the whole typically holds **pointers** to parts it did not create and **does not delete** them:

C++ · Department.cpp

```
#include <string>
#include <vector>
using namespace std;

class Professor {
    string name;
public:
    Professor(string n) : name(n) {}
};

class Department {
    string name;
    vector<Professor*> staff;          // pointers: Department does NOT own them
public:
    Department(string n) : name(n) {}
    void hire(Professor* p) { staff.push_back(p); }
    // no 'delete' in the destructor: professors outlive the department
};

int demo() {
    Professor ravi("Ravi");           // created OUTSIDE the department
    {
        Department cs("Computer Science");
        cs.hire(&ravi);
    }                                // department destroyed here...
    // ...ravi still exists
}
```

3.8 Composition (filled diamond)

Composition is a strong whole-part relationship: the whole **owns** the part, and the part's lifetime is tied to the whole. When the whole is destroyed, its parts are destroyed too. A part belongs to exactly **one** whole at a time. It is drawn with a **filled (black) diamond at the whole's end**.

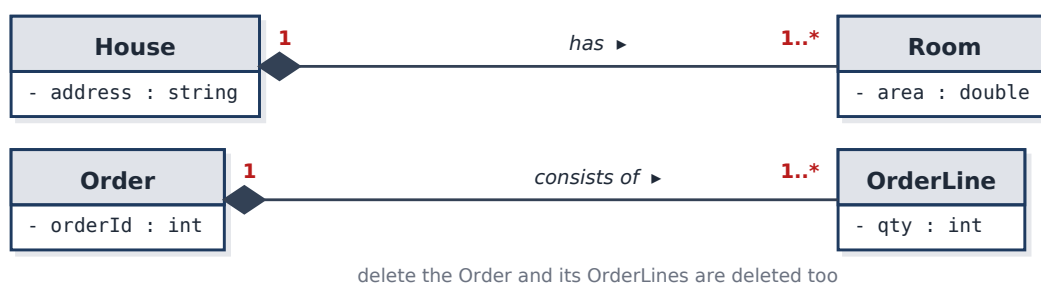


Figure 3.7 A house is composed of rooms; an order is composed of order lines.

A room cannot exist without its house, and an order line means nothing without its order. In C++, composition is most naturally a **member stored by value** (or a `unique_ptr`), so the compiler destroys the part along with the whole automatically:

C++ · House.cpp

```
#include <string>
#include <vector>
using namespace std;

class Room {
    double area;
public:
    Room(double a) : area(a) {}
};

class House {
    string address;
    vector<Room> rooms;           // stored BY VALUE: House owns its Rooms
public:
    House(string addr) : address(addr) {
        rooms.emplace_back(12.5); // rooms are created by the house
        rooms.emplace_back(9.0);
    }
};                               // destroy House -> Rooms destroyed too
```

Note

The multiplicity at the whole's end of a composition is always **1** (or **0..1**), because a part can belong to at most one whole. It is often left out because it is implied.

3.9 Aggregation vs composition

	Aggregation ◇	Composition ◆
Relationship	has-a (weak)	owns / is made of (strong)
Part's lifetime	Independent of the whole	Ends when the whole ends
Part shared?	Can be shared by several wholes	Belongs to exactly one whole
Who creates the part?	Usually created outside	Usually created by the whole
C++ typical form	Raw pointer / reference, not deleted by the whole	Member by value or unique_ptr
Examples	Department-Professor, Team-Player, Playlist-Song	House-Room, Order-OrderLine, Book-Page

The deletion test

Ask: **"If I delete the whole, should the parts disappear too?"**

- Yes → composition (filled diamond).
- No → aggregation (hollow diamond) or a plain association.

Also ask: "Can this part belong to two wholes at the same time?" If yes, it cannot be composition.

Note

The UML specification defines aggregation only loosely, so teams use it inconsistently. Many experienced designers draw a plain association instead of aggregation, and reserve diamonds for composition, where the meaning is precise. For exams, learn the textbook distinction above.

3.10 Generalization (inheritance)

Generalization shows an **is-a** relationship: the subclass inherits everything from the superclass and may add or override members. It is drawn as a **solid line with a hollow triangle pointing to the parent**. When several classes share a parent, the lines are usually merged into one arrowhead (a "tree").

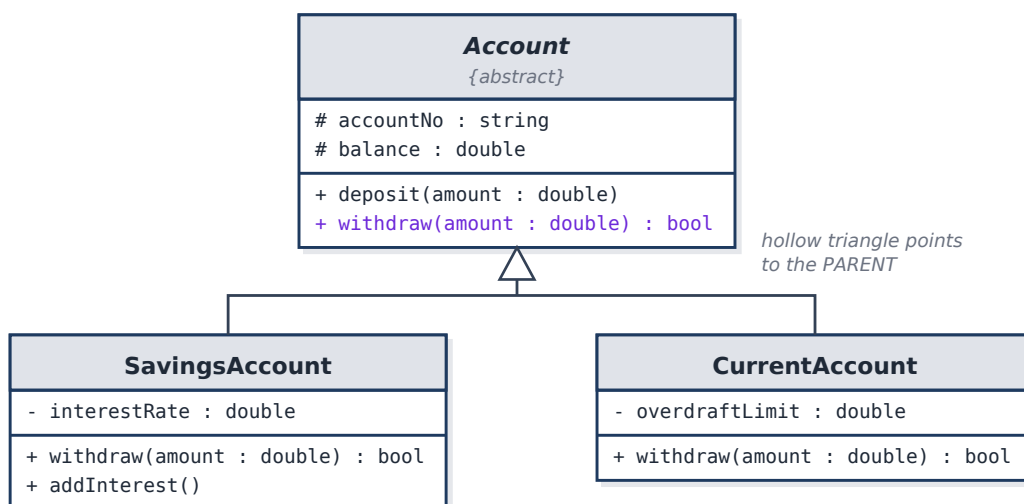


Figure 3.8 Two kinds of account inherit from the abstract class Account.

- **Account** is abstract (italic name, `{abstract}`) and `withdraw()` is abstract (italic): each subclass must provide its own version.
- Subclasses list **only what they add or override**. Do not repeat inherited members like `deposit()`; they are implied by the triangle.
- `#` (protected) attributes in the parent are accessible in the subclasses.

C++ · Account.cpp

```
#include <string>
using namespace std;

class Account {                                // {abstract}
protected:
    string accountNo;
    double balance = 0;
public:
    virtual ~Account() {}
    void deposit(double amount) { balance += amount; }
    virtual bool withdraw(double amount) = 0;    // italic = pure virtual
};

class SavingsAccount : public Account {        // hollow triangle
    double interestRate = 0.04;
public:
    bool withdraw(double amount) override {
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }
    void addInterest() { balance += balance * interestRate; }
};

class CurrentAccount : public Account {
    double overdraftLimit = 10000;
```

C++ · Account.cpp

```
public:
    bool withdraw(double amount) override {
        if (amount > balance + overdraftLimit) return false;
        balance -= amount;
        return true;
    }
};
```

3.11 Realization (implementing an interface)

Realization means a class **implements** an interface: it promises to provide every operation the interface declares. It is drawn as a **dashed line with a hollow triangle pointing to the interface**, like generalization but dashed.

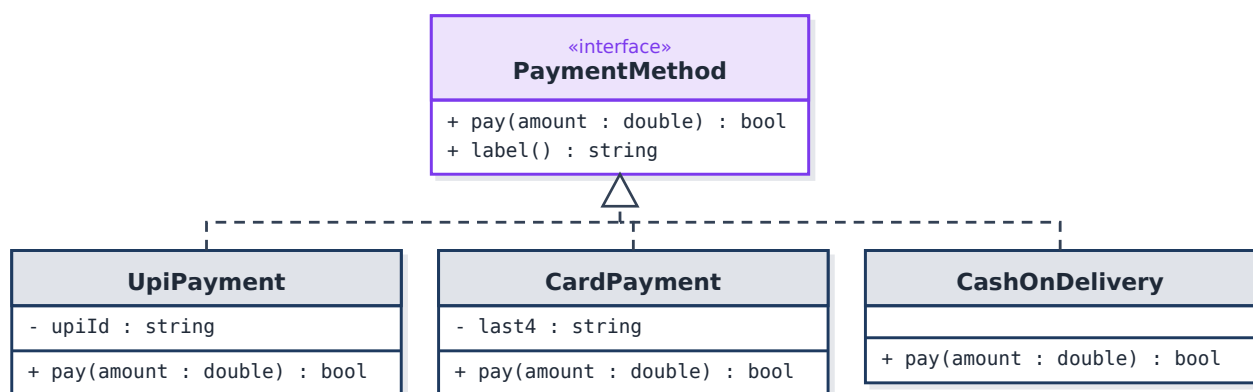


Figure 3.9 Three payment classes realize the PaymentMethod interface.

C++ · PaymentMethod.cpp

```
#include <string>
using namespace std;

class PaymentMethod { // <<interface>>
public:
    virtual ~PaymentMethod() {}
    virtual bool pay(double amount) = 0;
    virtual string label() const = 0;
};

class UpiPayment : public PaymentMethod { // dashed line + triangle
    string upiId;
public:
    UpiPayment(string id) : upiId(id) {}
    bool pay(double) override { return true; }
    string label() const override { return "UPI"; }
};
```

Note

You may also see the **lollipop** notation: a small circle on a stick attached to a class, labelled with the interface name. It means the same thing as realization, in a more compact form.

3.12 Dependency

A **dependency** is the weakest relationship: one class **uses** another temporarily, without keeping a long-term reference to it. If the used class changes, the dependent class may have to change too. It is drawn as a **dashed line with an open arrowhead** pointing to the class being used, optionally labelled with a stereotype such as `«use»`, `«create»` or `«call»`.



Figure 3.10 Dependencies: a class used as a parameter, and a class that is created and returned.

C++ · OrderService.cpp

```

class PaymentGateway {
public:
    bool charge(double) { return true; }
};

class OrderService {
    // No PaymentGateway member: OrderService only USES one briefly,
    // as a parameter. That is a dependency, not an association.
public:
    bool checkout(PaymentGateway& gw, double total) {
        return gw.charge(total);
    }
};
    
```

If class B appears in class A as...	Draw
A data member (stored for the object's lifetime)	Association (or aggregation / composition)
Only a function parameter, local variable or return type	Dependency
The base class of A	Generalization
An interface that A implements	Realization

3.13 Choosing the right relationship

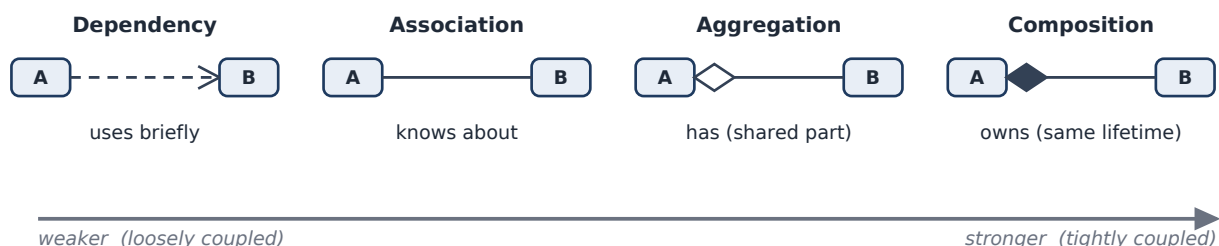


Figure 3.11 Relationship strength, from loosest to tightest coupling.

When you are unsure which line to draw, work through these questions in order and stop at the first "yes":

1. Is A a **kind of** B? → **Generalization**.
2. Does A **implement** interface B? → **Realization**.
3. Does A **own** B, so that B dies with A? → **Composition**.
4. Does A **have** B as a part that can live on its own or be shared? → **Aggregation** (or plain association).
5. Does A **keep a reference** to B as a member? → **Association** (add an arrow if only A knows about B).
6. Does A only **use** B inside a function? → **Dependency**.

Chapter 3 recap

- Association: solid line. Arrowhead = navigability. Label with name, roles, multiplicity.
- Multiplicity at an end = how many of THAT class per one of the other.
- Aggregation: hollow diamond at the whole; parts independent and shareable.
- Composition: filled diamond at the whole; parts die with the whole.
- Generalization: solid line, hollow triangle at the parent.
- Realization: dashed line, hollow triangle at the interface.
- Dependency: dashed line, open arrow to the class being used.

4. Drawing a Class Diagram Step by Step

Knowing the notation is half the job. The other half is turning a **written problem** into a diagram. This chapter walks through the standard method using a library system.

4.1 The problem statement

Requirements

A college **library** keeps a catalogue of **books**. Each book has an ISBN, a title and an author, and the library may own several physical **copies** of the same book.

Students and faculty register as **members**. A member can **borrow** copies; each **loan** records the issue date, due date and return date. Students may borrow up to 3 copies at a time, faculty up to 10.

If a copy is returned late, a **fine** is charged. The library wants to be able to change how fines are calculated in the future.

4.2 Step 1: find the candidate classes (nouns)

Underline every **noun** in the requirements. Each is a candidate class or attribute. Then filter: keep nouns that have their own data **and** behaviour; turn simple values into attributes; drop nouns that are outside the system or are duplicates.

Noun	Decision	Reason
library	Class Library	The system's top-level container
book	Class Book	Has data (ISBN, title, author)
copy	Class BookCopy	Each physical copy is borrowed separately and has its own availability
ISBN, title, author	Attributes of Book	Simple values with no behaviour
member	Abstract class Member	Common data for students and faculty
student, faculty	Subclasses of Member	Same data, different borrowing limits
loan	Class Loan	Has its own data (dates) and behaviour (fine)
issue / due / return date	Attributes of Loan	Simple values
fine	Interface FinePolicy	"May change in future" signals a replaceable rule: hide it behind an interface
college	Rejected	Outside the system

4.3 Step 2: attributes and operations (verbs)

Attributes come from the descriptive nouns (step 1). Operations come from **verbs**: *borrow*, *return*, *register*, *charge a fine*. Put each operation on the class that owns the data it needs. For example, `availableCopies()` belongs on **Book** because Book holds the list of copies; `fine()` belongs on **Loan** because Loan knows the dates.

4.4 Step 3: relationships

Pair	Question	Relationship
Student, Faculty → Member	Is a Student a kind of Member?	Generalization
Library → Book	Do books in the catalogue belong to the library record?	Composition
Book → BookCopy	Does a copy record make sense without its book?	Composition (no)
Library → Member	Do members still exist if the library record is removed?	Aggregation (people exist independently)
Member — Loan	Does a member keep track of their loans?	Association
Loan — BookCopy	Does a loan refer to one copy?	Association
Loan → FinePolicy	Does a loan only use the policy to calculate a fine?	Dependency
StandardFinePolicy → FinePolicy	Does it implement the interface?	Realization

4.5 Step 4: multiplicities

Apply the two-question method to every association: one library has 0..* books; one book has 1..* copies; one member has 0..* loans and each loan has exactly 1 borrower; one copy can appear in many loans over time (0..*), but each loan is for exactly 1 copy.

4.6 The finished diagram

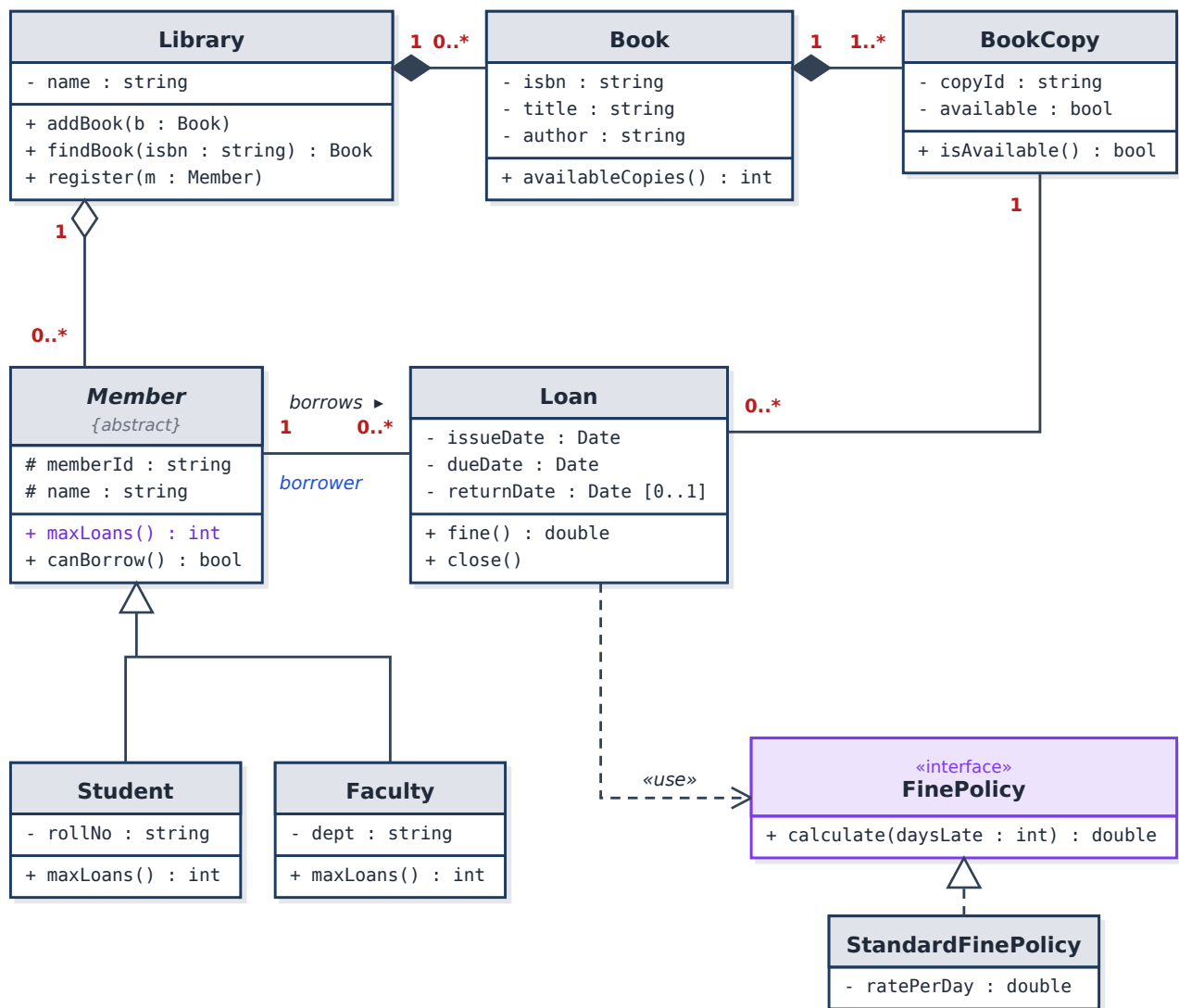


Figure 4.1 Class diagram for the library system.

Reading the diagram

- A Library **is composed of** 0..* Books; each Book **is composed of** 1..* BookCopies.
- A Library **aggregates** 0..* Members, who are either Students or Faculty (**Member** is abstract, and each subclass defines its own **maxLoans()**).
- A Member **borrows** through 0..* Loans; each Loan links exactly 1 borrower to exactly 1 BookCopy.
- A Loan **uses** a FinePolicy to compute **fine()**. Today that is StandardFinePolicy; a new policy can be added later without changing Loan.

4.7 The diagram as C++

Every box becomes a class and every line becomes a member, a base class or a parameter. Function bodies are kept minimal so the structure stays visible:

C++ · library.cpp

```

#include <string>
#include <vector>

```

C++ · library.cpp

```
#include <memory>
using namespace std;

class BookCopy {
    string copyId;
    bool available = true;
public:
    BookCopy(string id) : copyId(id) {}
    bool isAvailable() const { return available; }
};

class Book {
    string isbn, title, author;
    vector<BookCopy> copies; // composition 1 -- 1..*
public:
    Book(string i, string t, string a) : isbn(i), title(t), author(a) {}
    void addCopy(string id) { copies.emplace_back(id); }
    int availableCopies() const {
        int n = 0;
        for (const auto& c : copies) n += c.isAvailable();
        return n;
    }
};

class FinePolicy { // «interface»
public:
    virtual ~FinePolicy() {}
    virtual double calculate(int daysLate) const = 0;
};

class StandardFinePolicy : public FinePolicy { // realization
    double ratePerDay = 2.0;
public:
    double calculate(int daysLate) const override { return daysLate * ratePerDay; }
};

class Member; // forward declaration

class Loan {
    Member* borrower; // association to Member (1)
    BookCopy* copy; // association to BookCopy (1)
    string issueDate, dueDate, returnDate;
public:
    Loan(Member* m, BookCopy* c) : borrower(m), copy(c) {}
    double fine(const FinePolicy& policy, int daysLate) const { // «use»
        return policy.calculate(daysLate);
    }
};

class Member { // {abstract}
protected:
    string memberId, name;
    vector<Loan*> loans; // 1 -- 0..* Loan
public:
    virtual ~Member() {}
    virtual int maxLoans() const = 0;
    bool canBorrow() const { return (int)loans.size() < maxLoans(); }
};

class Student : public Member {
    string rollNo;
public:
    int maxLoans() const override { return 3; }
};
```

```
class Faculty : public Member {
    string dept;
public:
    int maxLoans() const override { return 10; }
};

class Library {
    string name;
    vector<unique_ptr<Book>> books;           // composition 1 -- 0..*
    vector<Member*> members;                 // aggregation 1 -- 0..*
public:
    void addBook(Book* b) { books.emplace_back(b); }
    void registerMember(Member* m) { members.push_back(m); }
};
```

Chapter 4 recap

- Nouns → classes or attributes. Verbs → operations.
- Put each operation on the class that owns the data it needs.
- Decide each relationship with the questions from 3.13.
- Add multiplicities with the two-question method.
- Look for "may change" or "different kinds of" in the requirements: they suggest interfaces and inheritance.

5. Worked Example: Online Shopping

A second complete diagram, this time for an online shop. It is the system whose checkout we will draw as a sequence diagram in Chapter 9, so the two parts of this guide connect.

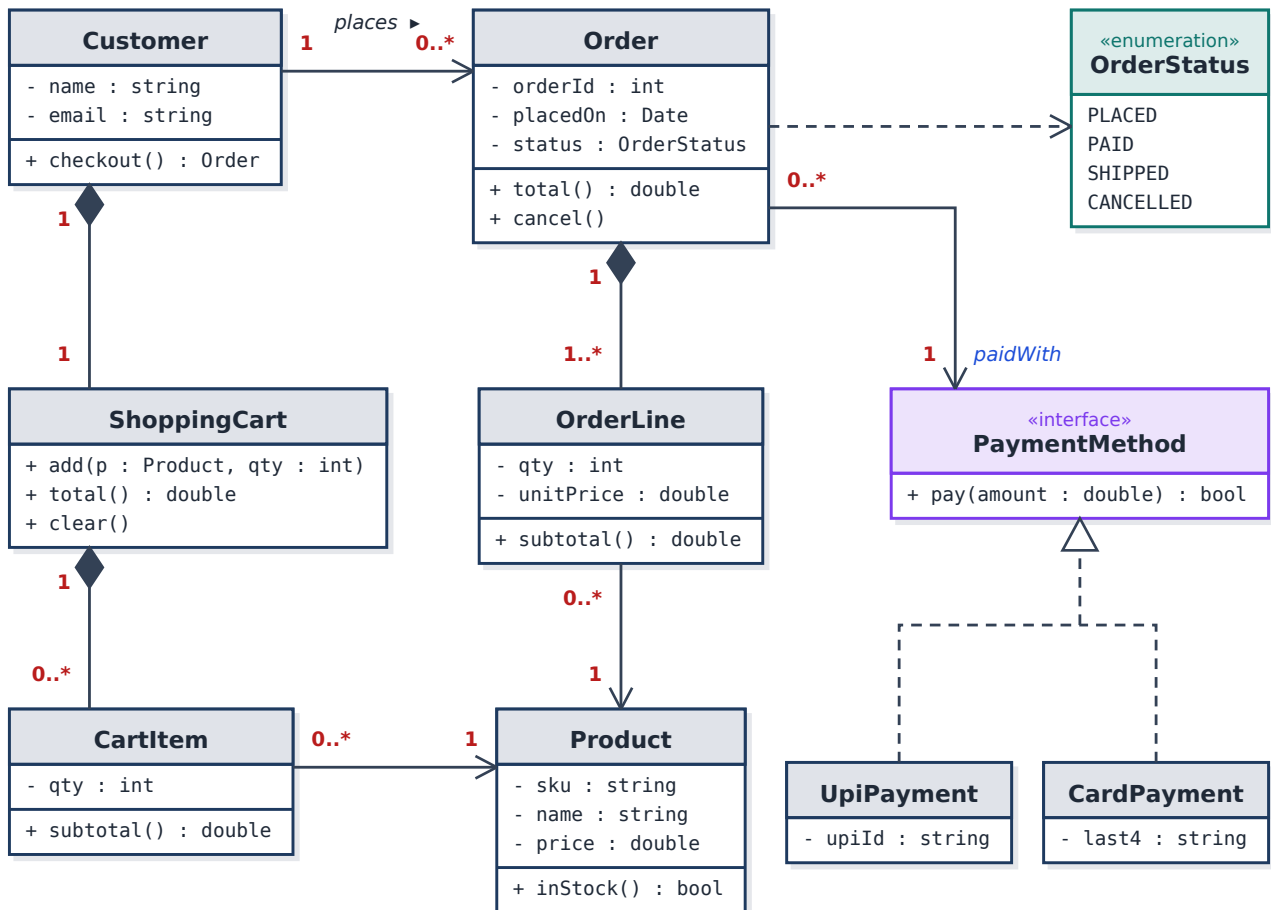


Figure 5.1 Class diagram for an online shop.

Relationship	Meaning	Why this kind
Customer → Order (1 to 0..*)	A customer places orders	Directed association: the customer keeps an order history; Order doesn't need to navigate back in this design
Customer ◆ ShoppingCart (1 to 1)	Each customer has one cart	Composition: the cart is meaningless without its customer
ShoppingCart ◆ CartItem (1 to 0..*)	A cart holds items	Composition: clearing the cart destroys its items
Order ◆ OrderLine (1 to 1..*)	An order has at least one line	Composition, and 1..* because an empty order is invalid
CartItem, OrderLine → Product	Each refers to one product	Directed association: the product does not need to know its carts or orders

Relationship	Meaning	Why this kind
Order → PaymentMethod (0..* to 1)	Each order is paid with one method	Association to the interface , so new payment types can be added freely
UpiPayment, CardPayment → PaymentMethod	Two implementations	Realization
Order → OrderStatus	Order uses the enumeration	Dependency (the status is also shown as an attribute)

Tip

Notice that **OrderLine** stores its own **unitPrice** instead of always reading the product's current price. If the product's price changes next week, old orders must still show what the customer actually paid. Class diagrams are where design decisions like this are made and discussed.

6. Class Diagram Tips and Common Mistakes

6.1 Common mistakes

Mistake	Why it is wrong	Fix
Triangle at the child	The arrow must point to the parent	Draw the triangle touching the superclass
Diamond at the part	The diamond marks the whole	Put it next to the owner (House, Order)
Multiplicity at the wrong end	Changes the meaning completely	Use the two-question method
Repeating inherited members in subclasses	Clutters the diagram; implied by inheritance	List only new or overridden members
Drawing an attribute AND an association for the same thing	Shows the same information twice	If it is a class with its own box, show a line, not an attribute
Everything public	Breaks encapsulation	Attributes private by default
Using inheritance for has-a	A Car is not an Engine	Use composition or association
Plural class names (Books)	A class describes ONE object	Use singular names
One giant diagram	Unreadable	Split into several focused diagrams

6.2 Good practice

- **Start simple.** Draw names and lines first; add attributes, types and multiplicities as the design firms up.
- **One diagram, one purpose.** A diagram explaining payments does not need every class in the system.
- **Avoid crossing lines** where possible; place tightly connected classes near each other and put parents above children.
- **Use names from the problem domain** (the words your users use), so non-programmers can read the diagram too.
- **Keep diagrams and code in sync**, or clearly mark a diagram as an early sketch.

6.3 Class diagrams as text: PlantUML

PlantUML lets you write a diagram as text and have the tool draw it. This is a simplified version of Figure 4.1:

PlantUML · library.puml

```

@startuml
class Library {
    - name : String
    + addBook(b : Book)
}
class Book {
    - isbn : String
    - title : String
    + availableCopies() : int
}
abstract class Member {
    # memberId : String
    {abstract} + maxLoans() : int
}
interface FinePolicy {
    + calculate(daysLate : int) : double
}

Library "1" *-- "0..*" Book          : composition
Library "1" o-- "0..*" Member        : aggregation
Book "1" *-- "1..*" BookCopy
Member <|-- Student                  : generalization
Member <|-- Faculty
Member "1" -- "0..*" Loan : borrows >
Loan "0..*" -- "1" BookCopy
Loan ..> FinePolicy : <<use>>
FinePolicy <|.. StandardFinePolicy  : realization
@enduml

```

PlantUML	Relationship
A -- B	Association
A --> B	Directed association
A o-- B	Aggregation (A is the whole)
A *-- B	Composition (A is the whole)
A < -- B	Generalization (B inherits from A)
A < .. B	Realization (B implements A)
A ..> B	Dependency

Part B · Sequence Diagrams

Participants, lifelines, messages and combined fragments: how objects collaborate over time.

7. Sequence Diagram Basics

A **sequence diagram** shows how a group of objects work together to carry out **one scenario**, by showing the **messages** they send each other in **time order**. Participants are arranged left to right across the top; time flows from **top to bottom**.

7.1 When to use one

- To design **how a use case or feature works** internally ("what happens when the user clicks Checkout?").
- To explain an **API or service interaction** (client, server, database, external payment provider).
- To understand **existing code** or debug a complex flow: drawing who calls whom often reveals the bug.
- In **system design interviews**, to walk through a request end to end.

Note

Draw one sequence diagram **per scenario**. Trying to show every possible path in one diagram makes it unreadable. Use fragments (Chapter 8) for small variations and separate diagrams for big ones.

7.2 Anatomy of a sequence diagram

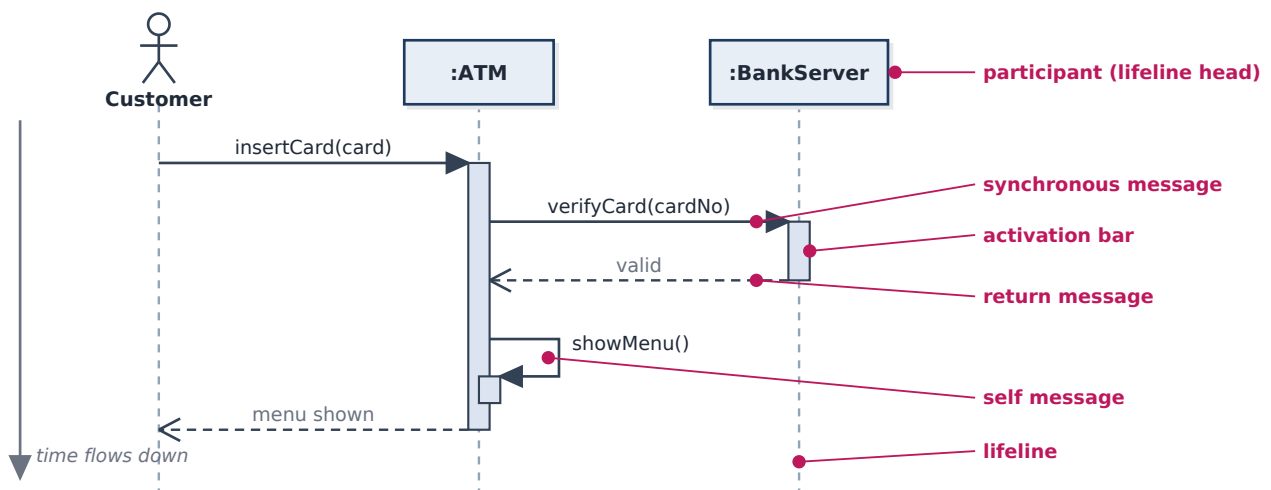


Figure 7.1 The basic elements of a sequence diagram.

7.3 Participants

Each participant is shown by a **head** (a box or a stick figure) at the top, with a dashed **lifeline** hanging below it.

Head label	Meaning
<u>acc</u> : Account	A named object acc of class Account
: Account	An anonymous object of class Account (the name does not matter)
acc	An object named acc , class not shown
Stick figure	An actor : a person or external system outside the software (Customer, Admin, Payment provider)

Note

In UML 1.x object names were underlined (acc : Account). UML 2 dropped the underline for lifelines. You will see both styles in textbooks; both are understood.

7.4 Lifelines and activation bars

- The **lifeline** (dashed vertical line) represents the participant existing over time.
- An **activation bar** (thin rectangle on the lifeline; the UML 2 name is *execution specification*) shows when the participant is **busy executing** an operation. It starts when a message arrives and ends when the work is done (usually when the reply is sent).
- If an object calls another object, its own bar continues while it waits, so bars show the **call stack**: who is waiting for whom.

7.5 Message types

A **message** is a horizontal arrow from the sender's lifeline to the receiver's lifeline. Usually it represents a method call. The arrow style says what kind of message it is:

Symbol	Name	Drawn as	Meaning
	Synchronous	Solid line, filled arrowhead	Sender waits for the receiver to finish (a normal function call)
	Asynchronous	Solid line, open arrowhead	Sender continues without waiting (event, queue message, background task)
	Return / reply	Dashed line, open arrowhead	Control (and maybe a value) goes back to the caller
	Self message	Arrow loops back to the same lifeline	An object calls one of its own operations
	Create	Dashed line, open arrowhead, to the new head	Creates a new object; its head appears at that height
	Destroy	Large X at the end of a lifeline	The object is destroyed (delete , or going out of scope)

Symbol	Name	Drawn as	Meaning
	Found message	Starts at a filled circle	Sender is unknown or outside the diagram's scope
	Lost message	Ends at a filled circle	Receiver is unknown, or the message never arrives

Message label syntax

UML · message syntax

```
[guard] seqNo: result = operationName(arguments)
```

Usually only `operationName(arguments)` is written. A **guard** in square brackets, such as `[balance > 0]`, means the message is sent only if the condition is true. **Sequence numbers** (1, 1.1, 1.2, 2) are optional in sequence diagrams, because vertical position already shows the order. `result` = names the returned value so later messages can use it.

7.6 Synchronous vs asynchronous

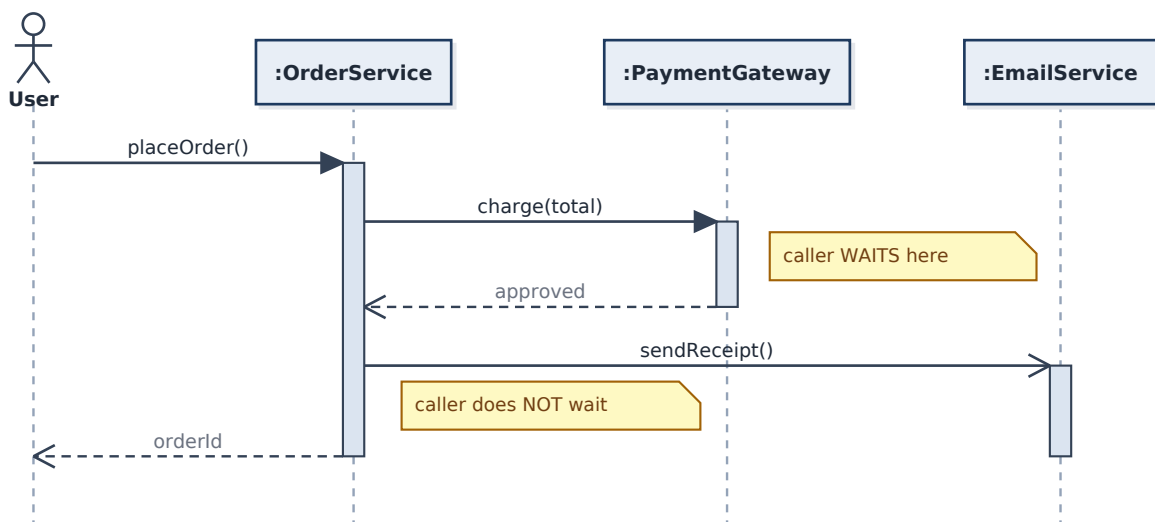


Figure 7.2 The service waits for the payment gateway (synchronous) but not for the email service (asynchronous).

The difference matters in real systems. A synchronous call to a slow service makes the whole request slow. Sending an email receipt **asynchronously** lets the customer get a response immediately while the email is sent in the background.

7.7 Return messages

Return messages are **optional** for synchronous calls, because the end of the activation bar already shows that control returned. Draw them when they **carry a meaningful value** (`orderId`, `balance`, `APPROVED`), and leave them out when the call returns nothing, to keep the diagram clean.

7.8 Self messages

When an object calls its own operation, the arrow loops back to the same lifeline, and a second, slightly offset activation bar shows the nested call. This is also how recursion is drawn.

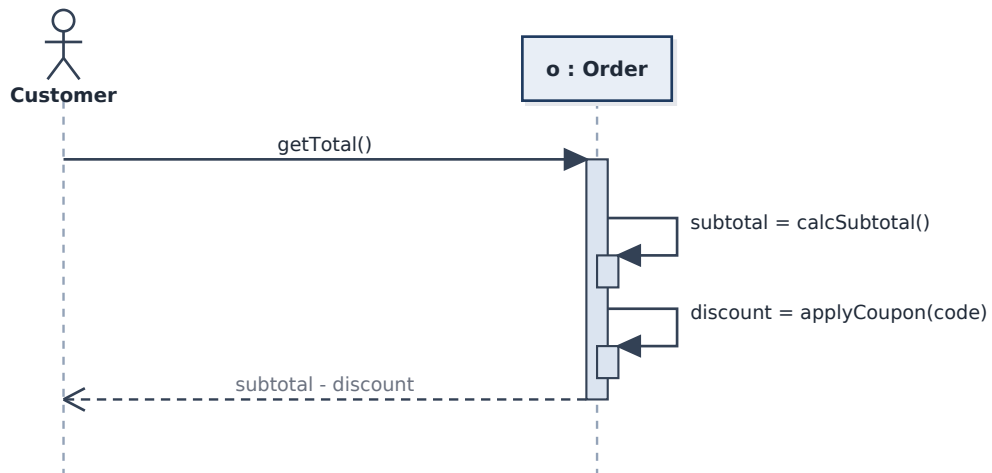


Figure 7.3 `getTotal()` calls two private helper operations on the same object.

7.9 Creating and destroying objects

A **create** message points to the **head** of the new participant, which is drawn lower down, at the moment of creation, instead of at the top. A **destroy** is shown by an **X** where the lifeline ends.

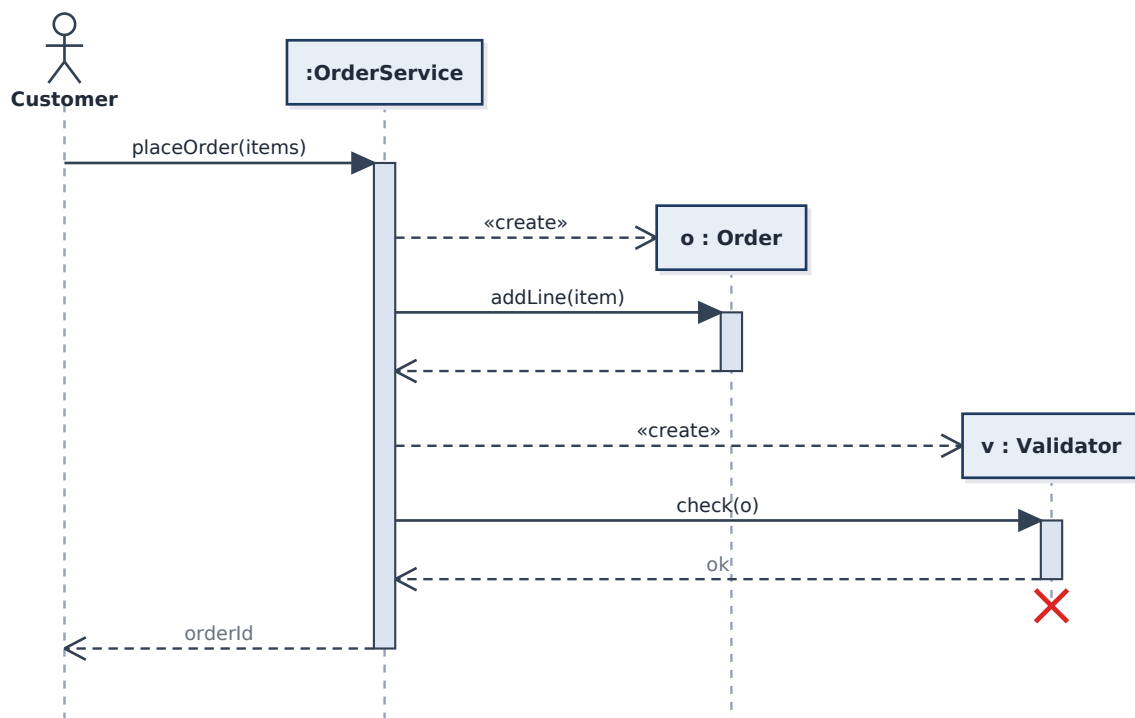


Figure 7.4 `OrderService` creates an `Order` and a temporary `Validator`; the `Validator` is destroyed after use.

In C++, the create message corresponds to a constructor call (`Order o;` or `new Order()`), and the X corresponds to the destructor running (the object going out of scope, or `delete`).

Chapter 7 recap

- Participants across the top; time flows down.
- Lifeline = dashed line; activation bar = the object is busy.
- Synchronous: filled arrowhead (caller waits). Asynchronous: open arrowhead.
- Return: dashed line; draw it when it carries a useful value.
- Create: dashed arrow to a lowered head. Destroy: X at the end of the lifeline.
- One diagram = one scenario.

8. Combined Fragments: Conditions, Loops and More

Real scenarios contain **if**, **else** and loops. Sequence diagrams show these with **combined fragments**: a rectangle drawn around part of the diagram, with an **operator** in a small pentagon at its top-left corner (**alt**, **opt**, **loop**, ...). A fragment can be split by dashed lines into **operands**, each with a **guard condition** in square brackets.

8.1 alt: if / else

alt (alternatives) chooses **exactly one** of its operands, the first whose guard is true. **[else]** covers every other case. It is the diagram form of **if ... else if ... else**.

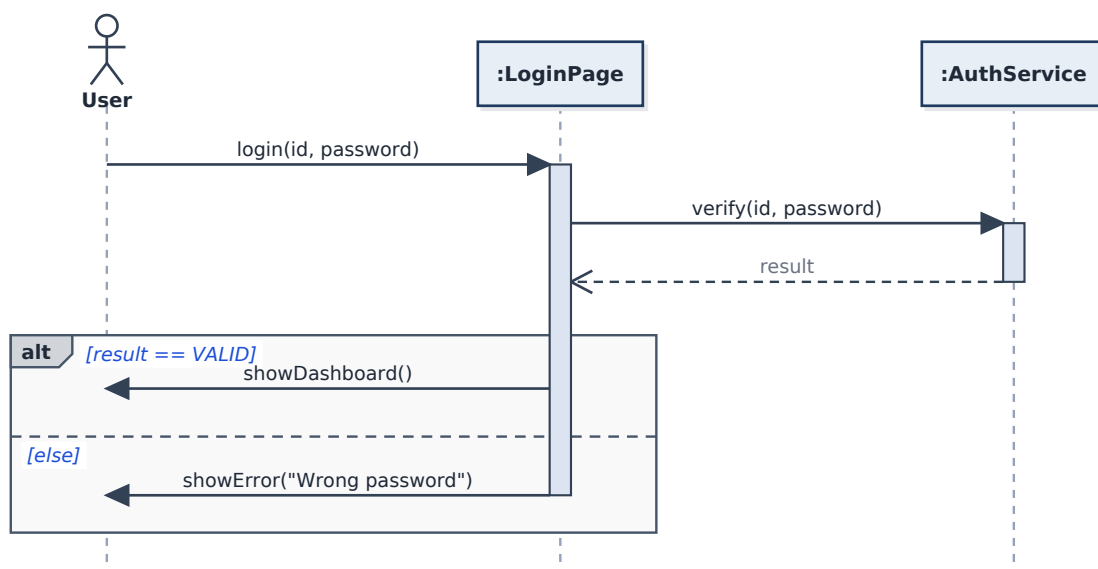


Figure 8.1 Login succeeds or fails.

C++ · alt as code

```

if (result == VALID) {
    page.showDashboard();
} else {
    page.showError("Wrong password");
}
  
```

8.2 opt: if without else

opt (option) has a **single operand** that runs only if its guard is true; otherwise nothing happens. It is the diagram form of an **if** with no **else**.

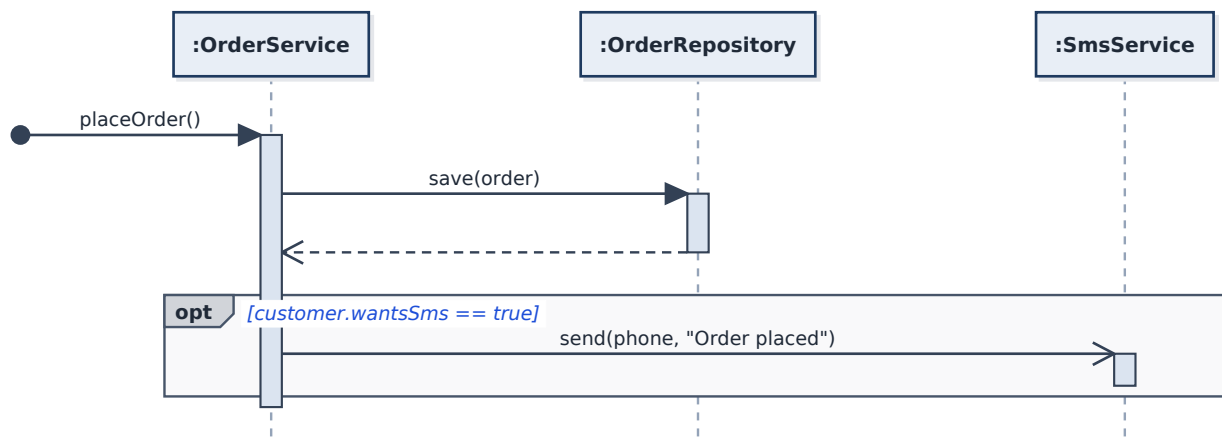


Figure 8.2 An SMS is sent only if the customer opted in.

8.3 loop: repetition

loop repeats its contents while the guard is true or once per element. You can also give minimum and maximum counts: **loop(3)** runs exactly 3 times, **loop(1, 5)** runs 1 to 5 times.

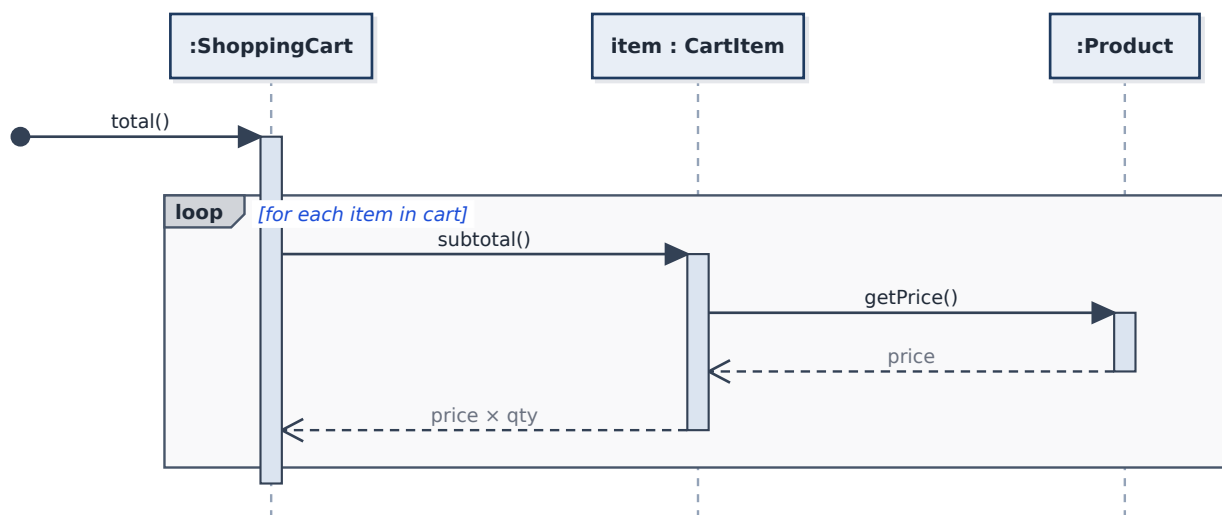


Figure 8.3 The cart asks every item for its subtotal.

C++ · loop as code

```

double ShoppingCart::total() {
    double sum = 0;
    for (CartItem& item : items) // loop [for each item in cart]
        sum += item.subtotal(); // -> item.subtotal() -> product.getPrice()
    return sum;
}
  
```

8.4 par: parallel

par means its operands **run in parallel**: there is no guaranteed order between them, although each operand's own messages stay in order. Use it for work done concurrently by threads or separate services.

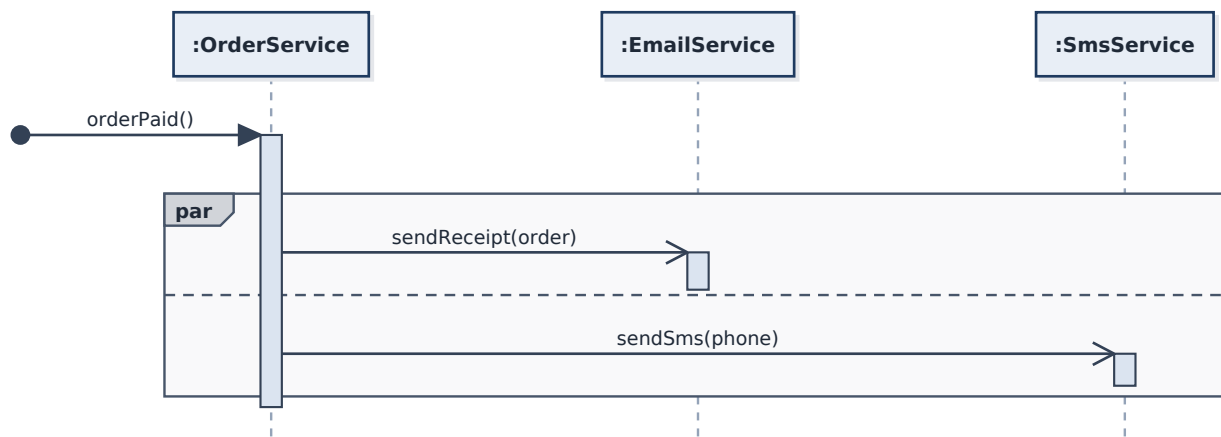


Figure 8.4 The receipt email and the SMS are sent at the same time.

8.5 break: early exit

break says: if the guard is true, run this fragment **and then leave the enclosing fragment** (skip the rest of it). It is the diagram form of **break** inside a loop, or of an early **return**.

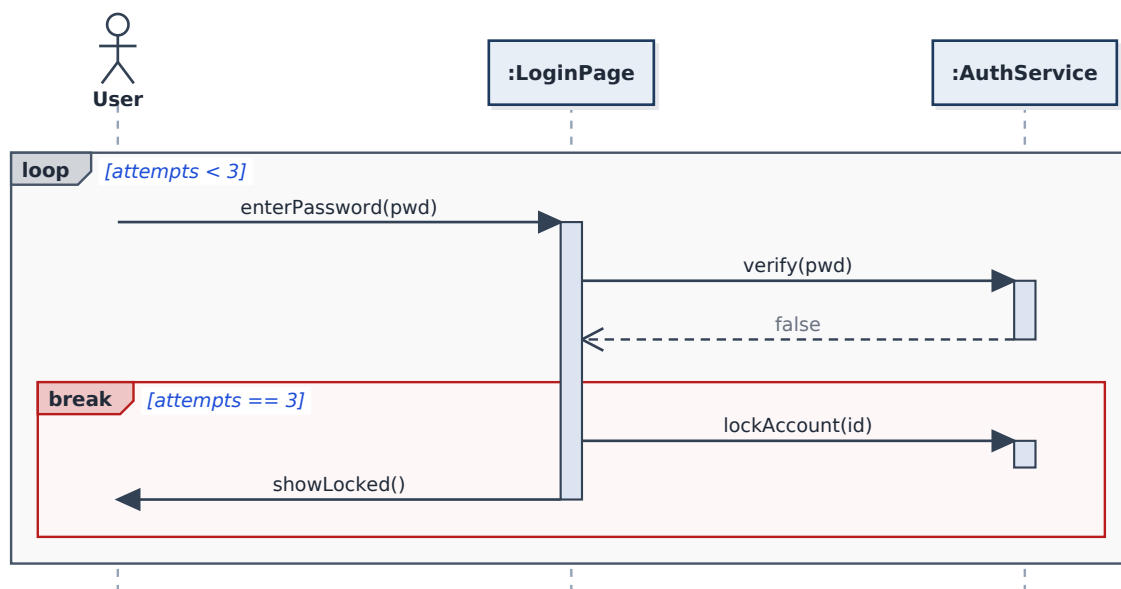


Figure 8.5 After three wrong passwords the loop is abandoned and the account is locked.

8.6 ref: reuse another diagram

A **ref** box (an **interaction use**) stands for a whole other sequence diagram, drawn across the lifelines it involves. It works like calling a function: draw the login steps once in a diagram named *Authenticate User*, then reference it wherever it is needed instead of repeating it.

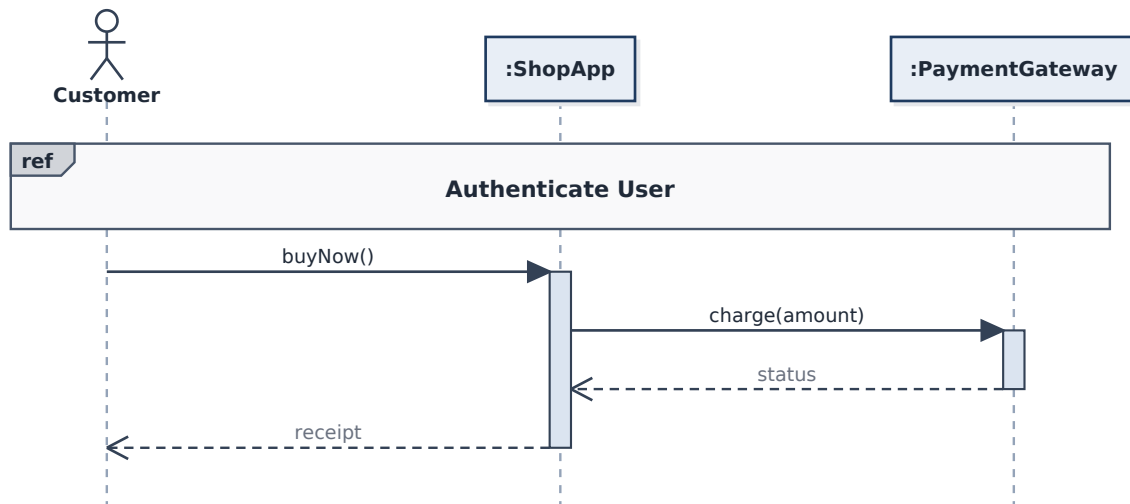


Figure 8.6 The authentication steps are detailed in a separate diagram.

8.7 All interaction operators

Operator	Meaning	Code equivalent
alt	Choose exactly one operand (first true guard)	if / else if / else, switch
opt	Run once if the guard is true	if without else
loop	Repeat while guard is true / for each / min..max times	for, while
par	Operands run in parallel	threads, async tasks
break	Run, then exit the enclosing fragment	break , early return
critical	Atomic region: nothing may interleave	a mutex-locked section
ref	Reference to another sequence diagram	calling a function
neg	Shows an invalid interaction that must never happen	(documentation)
assert	The only valid continuation	assert
seq / strict	Weak / strict ordering of operands	(rarely needed)
ignore / consider	Messages to ignore / consider	(rarely needed)

Tip

Fragments can be **nested**: a **break** inside a **loop** (Figure 8.5), or an **alt** inside a **loop**. Keep nesting to two levels; beyond that, split into separate diagrams with **ref**.

Chapter 8 recap

- A combined fragment = a box with an operator and guarded operands.
- **alt** = if/else, **opt** = if, **loop** = repetition, **par** = parallel, **break** = early exit, **ref** = reuse another diagram.
- Guards go in square brackets: **[balance >= amount]**, **[else]**.

9. Worked Examples

9.1 ATM cash withdrawal

Scenario

Scenario: A customer inserts a card, enters a PIN and asks to withdraw an amount. The ATM checks the PIN with the bank. The bank checks the account balance: if it is sufficient the money is deducted and dispensed; otherwise an error is shown. Finally the card is returned.

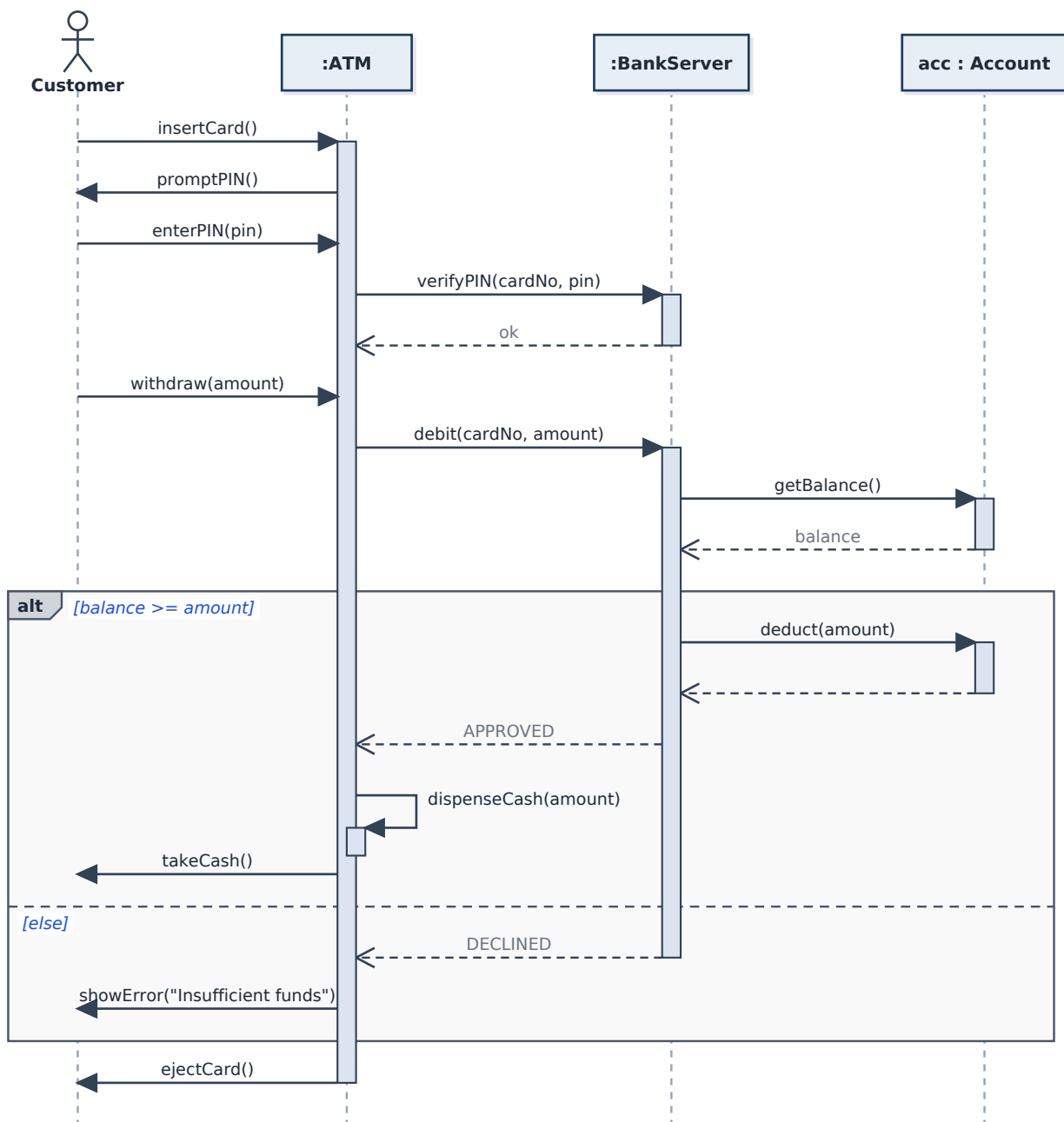


Figure 9.1 ATM withdrawal with an alt fragment for the balance check.

Walkthrough

1. The Customer (an actor) inserts a card. `:ATM` becomes active and stays active for the whole session, which is why its activation bar runs top to bottom.
2. The ATM prompts for the PIN, the customer enters it, and the ATM verifies it with `:BankServer` using a **synchronous** call: it must wait for the answer.
3. The customer requests a withdrawal. The ATM asks the bank to `debit()`, and the bank asks the `acc : Account` object for its balance.
4. The **alt** fragment has two operands. If `[balance >= amount]`, the bank deducts the money and replies APPROVED; the ATM calls its own `dispenseCash()` (a **self message**) and tells the customer to take the cash. `[else]`, the bank replies DECLINED and the ATM shows an error.
5. Whichever branch ran, the diagram continues below the fragment: the card is ejected.

9.2 Online checkout

Scenario

Scenario: A customer places an order from the checkout page. The order service reserves stock for every item, then charges the total through the payment gateway. If payment succeeds, an Order is created and marked paid, and the customer sees a confirmation. If it fails, the reserved stock is released and an error is shown.

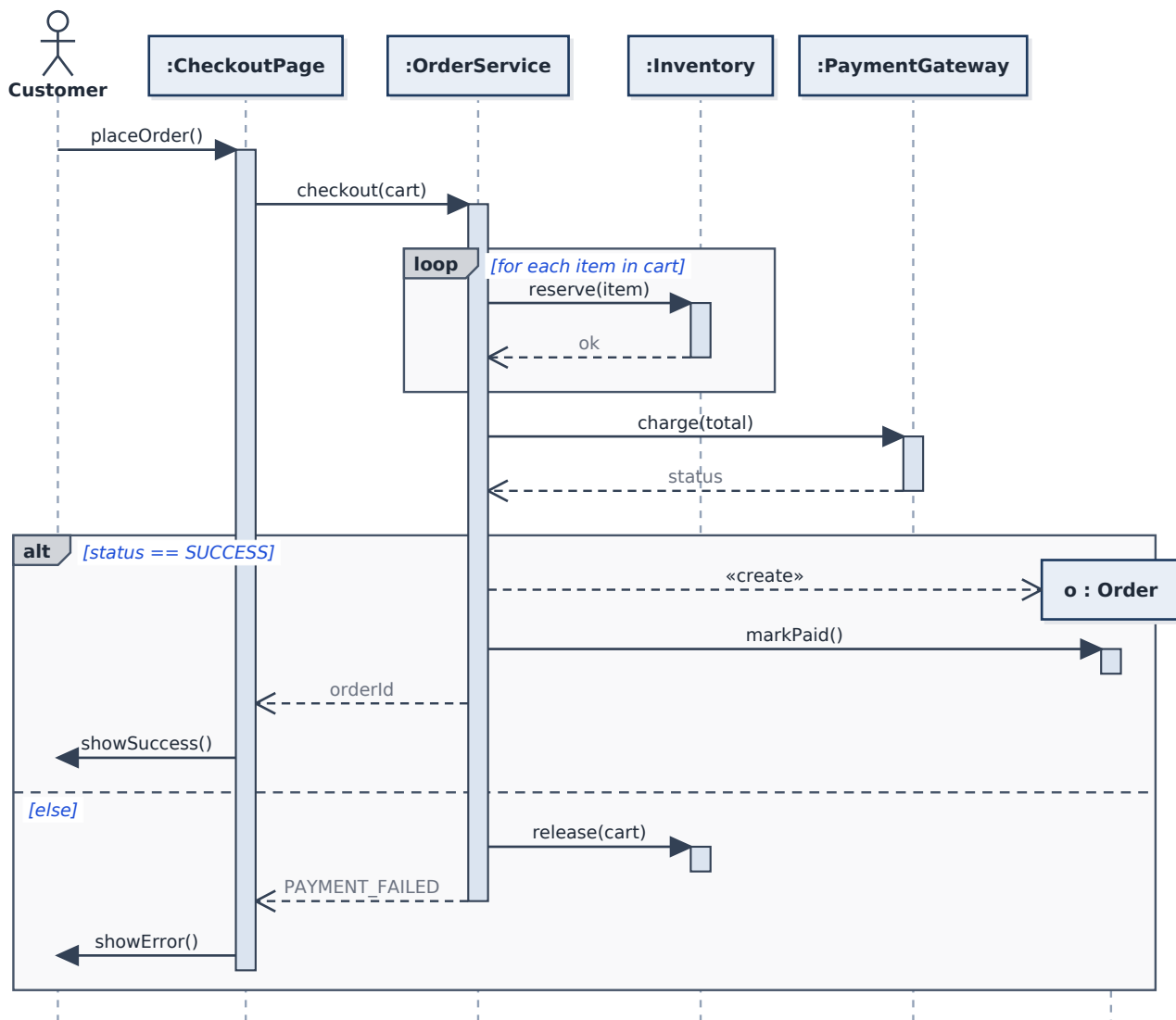


Figure 9.2 Checkout using loop, alt and a create message. The classes come from Figure 5.1.

From sequence diagram to C++

Every message in the diagram is a method call in the code below, and each activation bar is a method that is running. The program prints one line per message, so its output can be compared with the diagram line by line.

C++ · checkout.cpp

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Item { string sku; int qty; double price; };

class Inventory {
public:
    void reserve(const Item& it) {
        cout << "    OrderService -> Inventory : reserve(" << it.sku << ")\n";
    }
    void release() {
        cout << "    OrderService -> Inventory : release(cart)\n";
    }
};
  
```

C++ · checkout.cpp

```
    }
};

class PaymentGateway {
public:
    bool charge(double total) {
        cout << "    OrderService -> PaymentGateway : charge(" << total << ")\n";
        return total <= 5000;           // pretend card limit
    }
};

class Order {
public:
    Order() { cout << "    OrderService -> Order : <<create>>\n"; }
    void markPaid() { cout << "    OrderService -> Order : markPaid()\n"; }
    int id() const { return 1001; }
};

class OrderService {
    Inventory& inv;
    PaymentGateway& pay;
public:
    OrderService(Inventory& i, PaymentGateway& p) : inv(i), pay(p) {}

    // This whole method = the activation bar on :OrderService
    int checkout(const vector<Item>& cart) {
        double total = 0;
        for (const Item& it : cart) {           // loop [for each item in cart]
            inv.reserve(it);
            total += it.qty * it.price;
        }
        bool ok = pay.charge(total);           // synchronous call, waits

        if (ok) {                               // alt [status == SUCCESS]
            Order o;                             // create message
            o.markPaid();
            return o.id();                       // return message: orderId
        } else {                               // [else]
            inv.release();
            return -1;                          // return: PAYMENT_FAILED
        }
    }
};

class CheckoutPage {
    OrderService& svc;
public:
    CheckoutPage(OrderService& s) : svc(s) {}
    void placeOrder(const vector<Item>& cart) {
        cout << "Customer -> CheckoutPage : placeOrder()\n";
        cout << "    CheckoutPage -> OrderService : checkout(cart)\n";
        int id = svc.checkout(cart);
        if (id > 0) cout << "    CheckoutPage -> Customer : showSuccess() #" << id << "\n";
        else      cout << "    CheckoutPage -> Customer : showError()\n";
    }
};

int main() {
    Inventory inv;
    PaymentGateway gw;
    OrderService svc(inv, gw);
    CheckoutPage page(svc);

    cout << "--- Scenario 1: small cart ---\n";
```

C++ · checkout.cpp

```
page.placeOrder({{"PEN", 2, 50}, {"BOOK", 1, 450}});

cout << "\n--- Scenario 2: cart over the limit ---\n";
page.placeOrder({{"LAPTOP", 1, 55000}});
return 0;
}
```

OUTPUT

```
--- Scenario 1: small cart ---
Customer -> CheckoutPage : placeOrder()
CheckoutPage -> OrderService : checkout(cart)
OrderService -> Inventory : reserve(PEN)
OrderService -> Inventory : reserve(BOOK)
OrderService -> PaymentGateway : charge(550)
OrderService -> Order : <<create>>
OrderService -> Order : markPaid()
CheckoutPage -> Customer : showSuccess() #1001

--- Scenario 2: cart over the limit ---
Customer -> CheckoutPage : placeOrder()
CheckoutPage -> OrderService : checkout(cart)
OrderService -> Inventory : reserve(LAPTOP)
OrderService -> PaymentGateway : charge(55000)
OrderService -> Inventory : release(cart)
CheckoutPage -> Customer : showError()
```

Diagram element	C++
checkout(cart) message to :OrderService	svc.checkout(cart) call
Activation bar on :OrderService	The body of OrderService::checkout() running
loop [for each item in cart]	for (const Item& it : cart)
Synchronous charge(total) + return status	bool ok = pay.charge(total);
alt [status == SUCCESS] / [else]	if (ok) { ... } else { ... }
«create» to o : Order	Order o; (constructor runs)
Return orderId / PAYMENT_FAILED	return o.id(); / return -1;

Why this matters

Notice the two diagrams agree: every message sent to an object in Figure 9.2 is an operation of that object's class in Figure 5.1, or of a class closely related to it. Checking this agreement is how designers catch missing operations before writing code.

10. Drawing a Sequence Diagram Step by Step

10.1 The method

1. **Pick one scenario** and write it as a few sentences (like the boxes in Chapter 9). Choose the main success path first.
2. **Identify the participants**: the actor who starts it, and the objects involved. Take classes from your class diagram.
3. **Order them left to right** in the order they first get involved; the initiating actor goes on the far left. This keeps most arrows pointing right and avoids crossings.
4. **Draw the first message** from the actor, then follow the flow: for each step, ask "who does this, and whom do they need to ask?"
5. **Add activation bars** for each method execution, and **return messages** that carry useful values.
6. **Add fragments** for conditions (**alt**, **opt**) and repetition (**loop**).
7. **Check against the class diagram**: every message to an object must be an operation of its class, and the sender must have a way to reach the receiver (an association, a parameter or a local variable).

10.2 Common mistakes

Mistake	Fix
Trying to show every scenario in one diagram	One diagram per scenario; use alt/opt only for small variations
Messages that do not exist as operations in the class diagram	Add the operation to the class, or change the message
Using a filled arrowhead for everything	Open arrowhead for asynchronous; dashed for returns
Arrows pointing upward	Time flows down; every arrow is horizontal (or slightly downward for messages that take time)
Too much detail (getters, logging, every if)	Show the interactions that explain the design; skip trivial calls
Missing the actor or trigger	Every scenario starts somewhere: an actor or a found message
Mixing classes and objects in heads	Heads are objects: : Order , not just Order with a class box

10.3 Sequence diagrams as text: PlantUML

A simplified version of the ATM diagram (Figure 9.1) in PlantUML:

PlantUML · atm.puml

```
@startuml
actor Customer
participant ":ATM" as atm
participant ":BankServer" as bank
participant "acc : Account" as acc

Customer -> atm : withdraw(amount)
activate atm
atm -> bank : debit(cardNo, amount)
activate bank
bank -> acc : getBalance()
acc --> bank : balance

alt balance >= amount
    bank -> acc : deduct(amount)
    bank --> atm : APPROVED
    atm -> atm : dispenseCash(amount)
    atm -> Customer : takeCash()
else else
    bank --> atm : DECLINED
    atm -> Customer : showError("Insufficient funds")
end
deactivate bank
atm -> Customer : ejectCard()
deactivate atm
@enduml
```

PlantUML	Meaning
A -> B : msg()	Synchronous message
A ->> B : msg()	Asynchronous message
B --> A : value	Return message
A -> A : msg()	Self message
create B then A -> B : new	Create message
destroy B	Destroy (X)
activate B / deactivate B	Start / end an activation bar
alt ... else ... end	Alternatives (also opt, loop, par, break, critical, ref over A, B : name)

11. Class vs Sequence Diagrams

	Class diagram	Sequence diagram
Type	Structural (static)	Behavioral (interaction)
Shows	Classes, attributes, operations, relationships	Objects, messages, time order
Elements	Class boxes, 7 kinds of line, multiplicities	Lifelines, activation bars, messages, fragments
Scope	The whole system (or one area of it)	One scenario
Time	Not represented	Top to bottom
Answers	What exists? What does each class know and do?	How do objects work together to achieve this?
Code it maps to	Class declarations, members, inheritance	Method bodies: the calls inside them

Keeping the two consistent

- Every participant in a sequence diagram should be an object of a class in the class diagram (actors excepted).
- Every message sent to an object should be an **operation** of that object's class.
- If A sends a message to B, A must be able to reach B: through an association, a parameter, or by creating it. If not, a relationship is missing from the class diagram.
- New operations discovered while drawing sequence diagrams should be added to the class diagram. Designers usually go back and forth between the two.

12. Exam and Interview Questions

Q1. What is UML? Is it a programming language?

UML is a standard visual modeling language for specifying, visualising and documenting software. It is not a programming language and is independent of any language.

Q2. Name the two families of UML diagrams with examples.

Structural (class, object, component, deployment, package...) and behavioral (use case, activity, state machine, and interaction diagrams such as sequence and communication).

Q3. What are the three compartments of a class?

Name, attributes and operations.

Q4. What do +, -, # and ~ mean?

Public, private, protected and package visibility.

Q5. How are static and abstract members shown?

Static members are underlined; abstract classes and operations are in italics (or marked {abstract}).

Q6. What is multiplicity? Give examples.

The number of objects that can take part at one end of an association: 1, 0..1, 0..* (or), 1.., n..m.

Q7. Difference between association, aggregation and composition?

Association: objects are linked. Aggregation (hollow diamond): whole-part where parts are independent and can be shared. Composition (filled diamond): whole-part where the whole owns the parts and they are destroyed with it.

Q8. Where does the diamond go?

At the whole (the owner), never at the part.

Q9. Difference between generalization and realization?

Generalization: subclass inherits from a superclass (solid line, hollow triangle). Realization: a class implements an interface (dashed line, hollow triangle).

Q10. Difference between dependency and association?

Association is a lasting structural link (typically a member). Dependency is a temporary use: a parameter, local variable or return type (dashed arrow).

Q11. What is an association class?

A class attached to an association by a dashed line, holding data that belongs to the link (e.g. Enrollment with grade between Student and Course).

Q12. What is a lifeline? An activation bar?

A lifeline is the dashed line showing a participant over time. An activation bar shows the period during which the participant is executing an operation.

Q13. Synchronous vs asynchronous message?

Synchronous: sender waits for the reply (filled arrowhead). Asynchronous: sender continues immediately (open arrowhead).

Q14. How are object creation and destruction shown?

Creation: a dashed arrow to the head of the new lifeline, drawn at that point in time.

Destruction: an X at the end of the lifeline.

Q15. What are combined fragments? Name five.

Boxes that add control flow to a sequence diagram: alt (if/else), opt (if), loop, par (parallel), break; also critical, neg, ref.

Q16. alt vs opt?

alt chooses one of several operands (if/else); opt has one operand that runs only if its guard is true (if without else).

Q17. Are return messages compulsory?

No. They are optional for synchronous calls; draw them when they carry a meaningful value.

Q18. How do class and sequence diagrams relate?

Sequence diagram participants are objects of classes from the class diagram, and each message should correspond to an operation of the receiver's class.

13. Practice Exercises

Draw each diagram on paper first. Check your answer against the rules in this guide (especially the arrowhead and multiplicity rules) before looking anything up.

#	Exercise	Task	Hint
1	Hospital Class diagram	A hospital has departments. Each department employs doctors and nurses (both are staff). Patients are admitted to a ward and assigned one doctor. Draw the class diagram with multiplicities.	Staff is a superclass. Is Ward-Hospital composition?
2	University Class diagram	Students enroll in courses and receive a grade per course. Each course is taught by one professor; a professor teaches up to 4 courses.	Use an association class for the grade.
3	Social media Class diagram	Users create posts; posts have comments and likes; users follow other users.	Following is a reflexive association.
4	Spot the errors Class diagram	Draw: triangle at the subclass, diamond at the part, multiplicity on the wrong side. Then correct each one and explain.	Chapter 6.1
5	Food delivery order Sequence diagram	Customer places an order in the app; restaurant accepts; delivery partner is assigned; customer is notified.	Use async messages for notifications.
6	Library return Sequence diagram	A member returns a copy; if it is late, a fine is calculated using FinePolicy (Figure 4.1).	Use opt for the late case.
7	Online exam Sequence diagram	A student answers 20 questions, each saved to the server; at the end the result is calculated.	loop + self message
8	Both diagrams Class + sequence	For a movie-ticket booking system, draw a class diagram and a sequence diagram for "book 2 seats", and check they are consistent.	Use the checklist in Chapter 11.

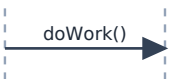
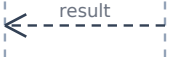
14. Cheat Sheet and Glossary

Class diagram notation

Symbol	Name	Remember
	Association	solid line: linked
	Directed association	arrow: who knows whom
	Aggregation	hollow diamond at whole; part independent
	Composition	filled diamond at whole; part dies with it
	Generalization	triangle at parent; is-a
	Realization	dashed + triangle at interface
	Dependency	dashed arrow: uses briefly

Item	Notation
Visibility	+ public - private # protected ~ package
Attribute	- name : Type [mult] = default {prop}
Operation	+ name(p : Type) : Return {query}
Static / abstract	underlined / <i>italic</i> or {abstract}
Multiplicity	1 0..1 * 0..* 1..* n..m
Special classes	«interface» «enumeration» {abstract}

Sequence diagram notation

Symbol	Name	Remember
	Synchronous	filled head: caller waits
	Asynchronous	open head: caller continues
	Return	dashed: reply / value
	Self message	calls own operation
	Create	dashed to a lowered head
	Destroy	X ends the lifeline

Fragment	Means
alt [c] / [else]	if / else
opt [c]	if
loop [c]	repeat
par	in parallel
break [c]	exit enclosing fragment
critical	atomic region
ref	see another diagram

Glossary

Term	Meaning
Activation bar	Thin rectangle on a lifeline: the participant is executing (execution specification).
Actor	A person or external system that interacts with the system (stick figure).
Aggregation	Weak whole-part association; parts can exist independently (hollow diamond).
Association	A structural link between classes (solid line).
Association class	A class holding data about an association link.
Combined fragment	A box adding control flow (alt, opt, loop...) to a sequence diagram.
Composition	Strong whole-part; parts die with the whole (filled diamond).
Dependency	One class temporarily uses another (dashed arrow).
Generalization	Inheritance; is-a (solid line, hollow triangle at parent).
Guard	A condition in square brackets controlling a message or fragment operand.
Lifeline	Dashed vertical line representing a participant over time.
Multiplicity	How many objects can participate at an association end.
Navigability	Which end of an association can be reached from the other (arrowhead).
Operand	One section of a combined fragment, separated by dashed lines.
Operation	A function declared on a class; its implementation is a method.
Realization	A class implements an interface (dashed line, hollow triangle).
Role name	Name for the part a class plays at one end of an association.
Stereotype	A «label» that extends UML with a specific meaning (e.g. «interface»).
Visibility	Access level of a member: + public, - private, # protected, ~ package.